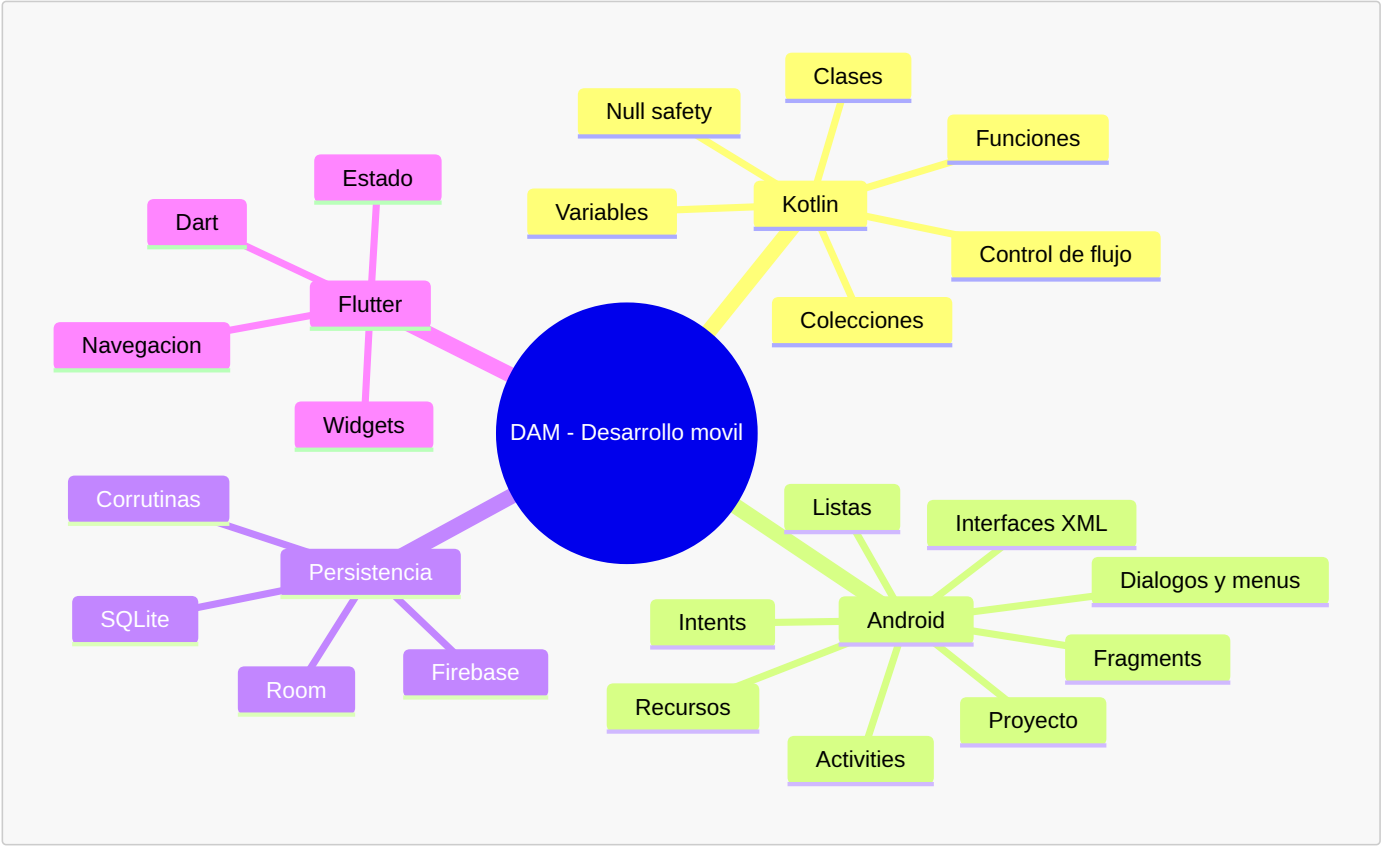


Apuntes de repaso: Android, Kotlin, Dart y Flutter

Archivo generado a partir del contenido teorico de [temario_completo](#) (temas 01, 02, 03, 04, 05, 07 y 08). Se omiten practicas y ejercicios.

Foco especial: ciclos de vida de [Activity](#) y [Fragment](#), acceso a recursos desde XML/Kotlin, sintaxis de Kotlin y sintaxis de Dart/Flutter.

Mapa general



1. Kotlin

1.1 Conceptos base

Concepto	Resumen	Ejemplo
Extension	Los archivos Kotlin terminan en <code>.kt</code> .	<code>MainActivity.kt</code>
Punto de entrada	La ejecucion empieza en <code>main</code> .	<code>fun main() { }</code>
<code>var</code>	Variable mutable.	<code>var edad = 20</code>
<code>val</code>	Referencia no reasignable.	<code>val PI = 3.14</code>
Inferencia	Kotlin deduce el tipo si puede.	<code>var nombre = "Ana"</code>
Visibilidad	<code>public</code> , <code>private</code> , <code>protected</code> , <code>internal</code> .	<code>private var contador = 0</code>
Sin <code>static</code> clasico	Se usan objetos, <code>companion object</code> o funciones a nivel de fichero.	<code>companion object { ... }</code>
Clases cerradas por defecto	Para heredar, la clase debe ser <code>open</code> .	<code>open class Persona</code>

1.2 Variables y null safety

Sintaxis	Significado	Uso recomendado
<code>String</code>	No puede ser <code>null</code> .	Datos obligatorios.
<code>String?</code>	Puede ser <code>null</code> .	Datos opcionales o no inicializados.

Sintaxis	Significado	Uso recomendado
<code>obj?.metodo()</code>	Llamada segura. Si <code>obj</code> es <code>null</code> , devuelve <code>null</code> .	Preferible frente a <code>!!</code> .
<code>obj!!</code>	Fuerza que no sea <code>null</code> ; si lo es, lanza excepcion.	Solo si se tiene certeza real.
<code>a ?: b</code>	Operador Elvis: si <code>a</code> es <code>null</code> , usa <code>b</code> .	Valores por defecto.
<code>lateinit var x</code>	Inicializacion tardia de propiedades no nulas.	Bindings, dependencias o datos inicializados despues.

```
var nombre: String? = null
println(nombre?.length ?: 0)

lateinit var texto: String
texto = "Inicializado despues"
```

1.3 Control de flujo

Estructura	Idea	Ejemplo corto
<code>if / else</code>	Igual que en Java, puede usarse en una linea.	<code>if (edad >= 18) "ok" else "no"</code>
<code>when</code>	Sustituye a <code>switch</code> ; permite rangos y condiciones.	<code>when(nota) { in 5..10 -> "aprobado" }</code>
<code>for</code> con rango	Recorre valores.	<code>for (i in 1..5)</code>
<code>step</code>	Salto entre valores.	<code>for (i in 0..10 step 2)</code>
<code>downTo</code>	Rango descendente.	<code>for (i in 10 downTo 1)</code>
<code>forEach</code>	Iteracion funcional.	<code>lista.forEach { println(it) }</code>
<code>forEachIndexed</code>	Itera con indice.	<code>lista.forEachIndexed { i, v -> ... }</code>

```
when (nota) {
    !in 0..10 -> "Nota incorrecta"
    in 5..9 -> "Aprobado"
    10 -> "Matricula"
    else -> "Suspenso"
}
```

1.4 Funciones

Tipo	Sintaxis	Comentario
Sin retorno	<code>fun saludar() { }</code>	Retorna <code>Unit</code> .
Con retorno	<code>fun suma(a: Int, b: Int): Int</code>	El tipo va despues de los parametros.
Una linea	<code>fun suma(a: Int, b: Int) = a + b</code>	Retorno inferido.
Parametros nombrados	<code>registrar(nombre = "Ana")</code>	Mejora legibilidad.
Valores por defecto	<code>fun f(x: Int = 0)</code>	Permite omitir argumentos.
Lambda	<code>{ a: Int, b: Int -> a + b }</code>	Muy usada en listeners y colecciones.

```
fun sumar(a: Int, b: Int = 0): Int = a + b

val operacion: (Int, Int) -> Int = { a, b -> a + b }
```

1.5 Arrays y colecciones

Elemento	Creacion	Uso
Array vacio	<code>emptyArray<String>()</code>	Sin elementos iniciales.
Array con nulos	<code>arrayOfNulls<String>(5)</code>	Tamano fijo con posiciones nulas.
Array con datos	<code>arrayOf(1, 2, 3)</code>	Puede inferir tipo.
Array primitivo	<code>intArrayOf(1, 2, 3)</code>	Optimizado para <code>Int</code> .
<code>ArrayList</code>	<code>ArrayList<String>()</code>	Lista mutable.
Busqueda	<code>find { ... }</code>	Devuelve el primer elemento.
Filtro	<code>filter { ... }</code>	Devuelve todos los que cumplen.
Transformacion	<code>map { ... }</code>	Genera otra coleccion.

```
val lenguajes = arrayOf("Java", "Kotlin", "Dart")
val encontrado = lenguajes.find { it.contains("Kot") }
val largos = lenguajes.filter { it.length > 4 }
```

1.6 Clases, data class, herencia e interfaces

Concepto	Kotlin	Nota
Clase simple	<code>class Persona</code>	Constructor vacio implicito.
Constructor primario	<code>class Persona(val nombre: String)</code>	En la firma de clase.
Constructor secundario	<code>constructor(...) : this(...)</code>	Debe delegar en el primario si existe.
Bloque inicial	<code>init { ... }</code>	Se ejecuta al construir.
Clase de datos	<code>data class Usuario(...)</code>	Ideal para modelos.
Herencia	<code>class Alumno : Persona()</code>	La superclase debe ser <code>open</code> .
Sobrescritura	<code>override fun metodo()</code>	El metodo de origen debe ser <code>open</code> .
Interfaz	<code>interface Listener { fun onClick() }</code>	Obliga a implementar metodos.

```
open class Persona(open val nombre: String) {
    open fun mostrar() = println(nombre)
}

data class Usuario(val id: Long, val email: String)

class Trabajador(
    override val nombre: String,
    val numeroSS: Int
) : Persona(nombre) {
    override fun mostrar() = println("$nombre - $numeroSS")
}
```

2. Android: proyecto, recursos y XML-Kotlin

2.1 Componentes basicos

Componente	Para que sirve	Se declara normalmente en
<code>Activity</code>	Pantalla con interfaz grafica.	Kotlin + <code>AndroidManifest.xml</code> + layout XML.
<code>View</code>	Elemento grafico: <code>TextView</code> , <code>Button</code> , <code>EditText</code> , etc.	<code>res/layout</code> .
<code>Service</code>	Trabajo en segundo plano sin UI directa.	Kotlin + manifest.

Componente	Para que sirve	Se declara normalmente en
<code>ContentProvider</code>	Intercambio de datos entre apps.	Kotlin + manifest.
<code>BroadcastReceiver</code>	Recibe eventos del sistema u otras apps.	Kotlin + manifest.
<code>Intent</code>	Navegacion o accion del sistema.	Codigo Kotlin.

2.2 Estructura de proyecto Android

Ruta	Contenido	Ejemplos
<code>AndroidManifest.xml</code>	Punto de entrada, activities, permisos, servicios.	<code>MAIN</code> , <code>LAUNCHER</code> , <code>uses-permission</code> .
<code>kotlin/paquete</code>	Logica de la app.	<code>MainActivity.kt</code> , adaptadores, modelos.
<code>res/layout</code>	Interfaces XML.	<code>activity_main.xml</code> , <code>fila_recycler.xml</code> .
<code>res/drawable</code>	Imagenes y XML graficos escalables.	PNG, vectores, shapes.
<code>res/mipmap</code>	Iconos de launcher por densidad.	<code>ic_launcher</code> .
<code>res/values</code>	Constantes reutilizables.	<code>strings.xml</code> , <code>colors.xml</code> , <code>dimens.xml</code> , <code>styles.xml</code> .
<code>res/menu</code>	Menus de opciones/contextuales.	<code>menu_principal.xml</code> .

2.3 Acceso a recursos desde XML y Kotlin

Recurso	Definicion	Desde XML	Desde Kotlin
String	<code><string name="saludo">Hola</string></code>	<code>@string/saludo</code>	<code>getString(R.string.saludo)</code> o <code>R.string.saludo</code> segun metodo.
Color	<code><color name="primario">#2196F3</color></code>	<code>@color/primario</code>	<code>getColor(R.color.primario)</code> o <code>ContextCompat.getColor(...)</code> .
Dimension	<code><dimen name="margen">16dp</dimen></code>	<code>@dimen/margen</code>	<code>resources.getDimension(R.dimen.margen)</code> .
Drawable	<code>res/drawable/logo.png</code>	<code>@drawable/logo</code>	<code>setImageResource(R.drawable.logo)</code> .
Layout	<code>activity_main.xml</code>	Incluido por nombre	<code>setContentView(R.layout.activity_main)</code> o <code>binding</code> .
Menu	<code>res/menu/menu_principal.xml</code>	No aplica directamente	<code>menuInflater.inflate(R.menu.menu_principal, menu)</code> .
Array	<code><string-array name="paises">...</string-array></code>	<code>@array/paises</code>	<code>resources.getStringArray(R.array.paises)</code> .

```

<!-- res/values/strings.xml -->
<resources>
    <string name="boton_saludar">Saludar</string>
    <string name="texto_defecto">Texto por defecto</string>
</resources>

```

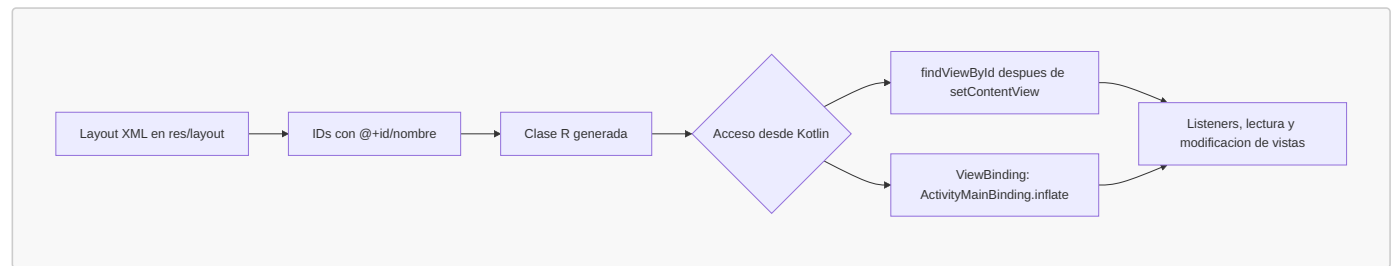
```

<Button
    android:id="@+id/boton_saludar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/boton_saludar" />

```

```
binding.textoCambiar.text = getString(R.string.texto_defecto)
binding.imagenLogo.setImageResource(R.drawable.logo)
```

2.4 Flujo XML - Kotlin



Forma	Ventaja	Riesgo / detalle
<code>findViewById<T>(R.id.id)</code>	Directo y clasico.	Debe ejecutarse despues de <code>setContentView</code> ; mas propenso a errores.
<code>ViewBinding</code>	Tipado, rapido, menos nulos.	Hay que activarlo en Gradle y usar la clase generada.

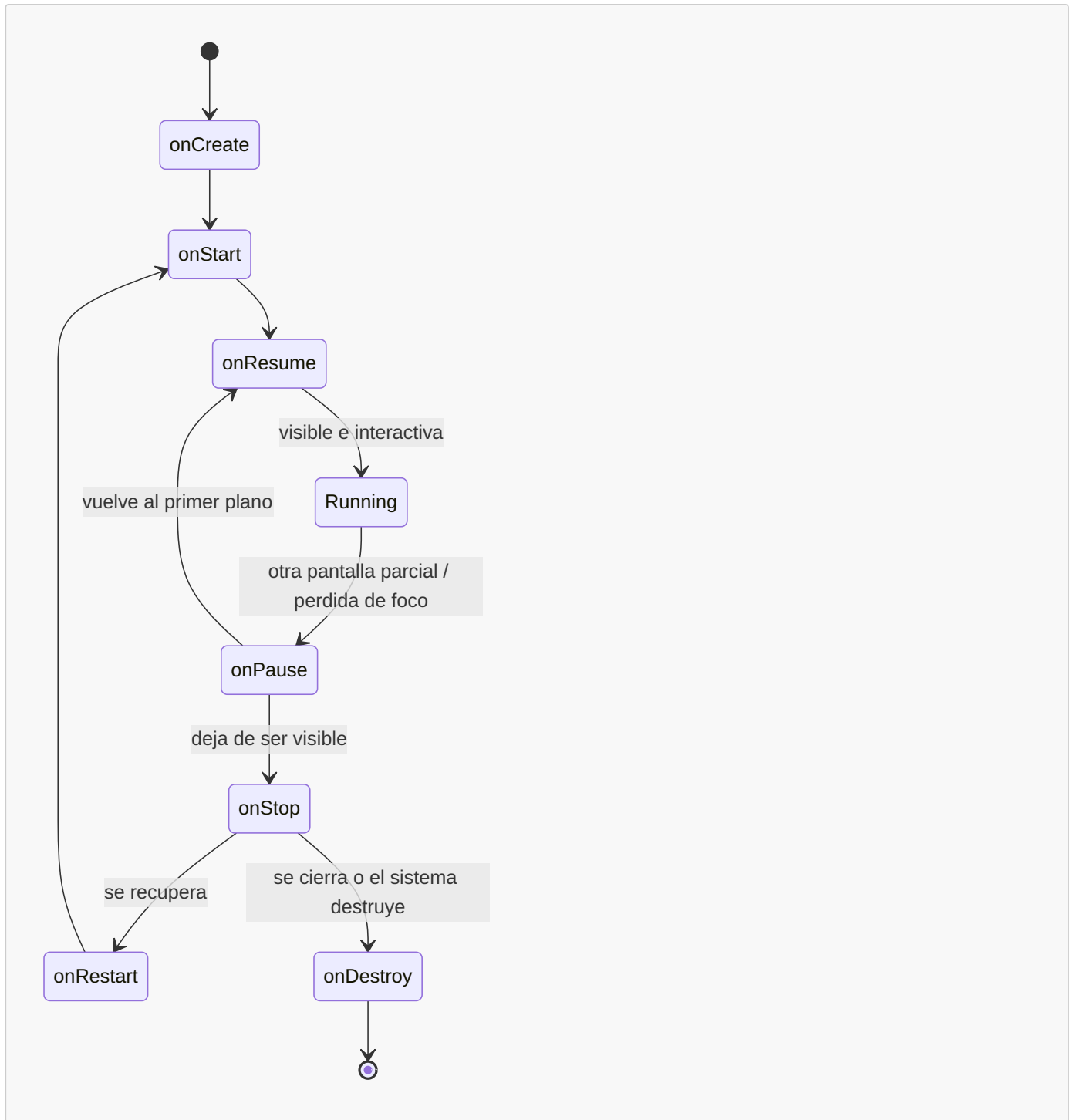
```
private lateinit var binding: ActivityMainBinding

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    binding.botonPulsar.setOnClickListener {
        binding.textoCambiar.text = binding.editCapturar.text.toString()
    }
}
```

3. Ciclo de vida de una Activity

3.1 Esquema Mermaid



3.2 Metodos principales

Metodo	Cuando ocurre	Uso tipico
<code>onCreate()</code>	Se crea la activity.	Inflar layout, binding, inicializar estado y listeners.
<code>onStart()</code>	La pantalla empieza a ser visible.	Preparar recursos visibles.
<code>onResume()</code>	La pantalla ya es interactiva.	Reanudar sensores, camara, animaciones o listeners activos.
<code>onPause()</code>	Pierde foco parcialmente.	Guardar cambios ligeros, pausar animaciones.
<code>onStop()</code>	Ya no es visible.	Liberar recursos pesados visibles.
<code>onRestart()</code>	Vuelve tras haber estado parada.	Repreparar lo necesario antes de <code>onStart</code> .
<code>onDestroy()</code>	Se destruye.	Limpieza final, cancelar referencias si procede.
<code>onSaveInstanceState()</code>	Antes de recreacion por configuracion.	Guardar estado pequeno en <code>Bundle</code> .

3.3 Cambios de configuracion

Cuando cambia la orientacion, idioma u otra configuracion, Android puede destruir y recrear la **Activity**. Por eso, si inicializas un contador a 0 en **onCreate**, volvera a 0 tras girar la pantalla si no guardas estado.

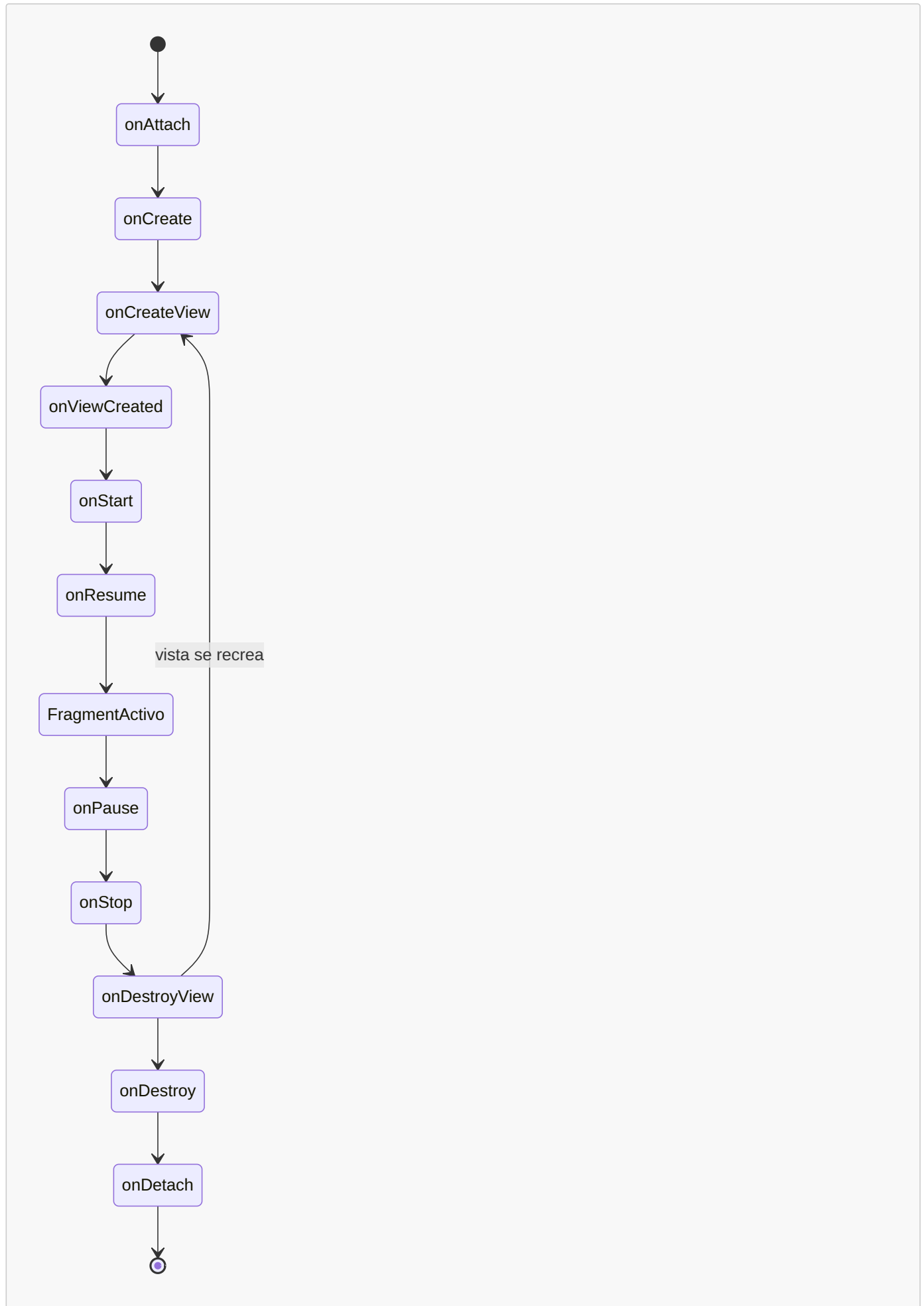
```
override fun onSaveInstanceState(outState: Bundle) {  
    super.onSaveInstanceState(outState)  
    outState.putInt("contador", contador)  
}  
  
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    contador = savedInstanceState?.getInt("contador") ?: 0  
}
```

Problema	Solucion
Layout vertical no sirve en horizontal.	Crear otro <code>activity_main.xml</code> con cualificador <code>land</code> .
Datos se reinician.	Guardar en <code>onSaveInstanceState</code> y recuperar en <code>onCreate</code> .
Una vista solo existe en landscape.	Acceder con null safety: <code>binding.buttonReset?.setOnClickListener(...)</code> .
No quieres recreacion automatica.	Declarar <code>android:configChanges</code> , pero gestionarlo manualmente.

4. Ciclo de vida de un Fragment

En los apuntes disponibles aparece **DialogFragment** dentro de dialogs; su ciclo de vida se basa en el de fragments. Este resumen recoge la secuencia estandar de fragments Android.

4.1 Esquema Mermaid



4.2 Metodos principales

Metodo	Que pasa	Uso tipico
<code>onAttach(context)</code>	El fragment se asocia a una activity/contexto.	Obtener callbacks de la activity, validar interfaces.
<code>onCreate()</code>	Se crea el fragment, aun sin vista.	Inicializar datos no dependientes de UI.
<code>onCreateView()</code>	Se infla el layout del fragment.	<code>inflater.inflate(...)</code> o binding del fragment.
<code>onViewCreated()</code>	La vista ya existe.	Inicializar vistas, listeners, RecyclerView.
<code>onStart()</code>	El fragment es visible.	Preparar elementos visibles.
<code>onResume()</code>	El fragment es interactivo.	Reanudar interaccion.
<code>onPause()</code>	Pierde foco.	Pausar acciones.
<code>onStop()</code>	Deja de ser visible.	Detener trabajo visual.
<code>onDestroyView()</code>	Se destruye la vista, no necesariamente el fragment.	Limpiar binding: <code>_binding = null</code> .
<code>onDestroy()</code>	Se destruye el fragment.	Liberar datos propios.
<code>onDetach()</code>	Se separa de la activity.	Soltar referencias al contexto.

4.3 Fragment vs Activity

Aspecto	Activity	Fragment
Representa	Pantalla completa o entrada principal.	Parte reutilizable de UI dentro de una activity.
Layout	<code>setContentView(...)</code> .	<code>onCreateView(...)</code> .
Contexto	Es un <code>Context</code> .	Necesita <code>requireContext()</code> o <code>context</code> .
Navegacion	<code>Intent</code> , Navigation Component, etc.	Fragment transactions o Navigation Component.
Ciclo de vida de vista	No separado.	Separado: fragment y su view no mueren siempre a la vez.

```
class MiFragment : Fragment(R.layout.fragment_mi) {
    private var _binding: FragmentMiBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        _binding = FragmentMiBinding.bind(view)
        binding.boton.setOnClickListener { /* accion */ }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

5. Interfaz grafica Android

5.1 Layouts

Layout	Uso	Claves
<code>LinearLayout</code>	Colocar elementos en fila o columna.	<code>orientation</code> , <code>layout_weight</code> , <code>gravity</code> .
<code>ConstraintLayout</code>	Posicion relativa entre elementos o parent.	<code>app:layout_constraint...</code>
<code>FrameLayout</code>	Apilar o alojar fragments.	Suele usarse como contenedor.
<code>RecyclerView</code>	Listas eficientes y grandes.	Requiere adapter + holder + layout manager.

5.2 Vistas basicas y eventos

Vista	XML habitual	Evento / lectura
Button	<Button ... />	setOnClickListener { ... }
EditText	hint, inputType	binding.edit.text.toString()
CheckBox	Estado booleano	isChecked, setOnCheckedChangeListener
RadioButton	Dentro de RadioGroup	RadioGroup.setOnCheckedChangeListener
Spinner	Desplegable	onItemSelectedListener
AutoCompleteTextView	Campo con sugerencias	ArrayAdapter

```
binding.grupoRadios.setOnCheckedChangeListener { _, checkedId ->
    when (checkedId) {
        R.id.radio_uno -> Snackbar.make(binding.root, "Uno", Snackbar.LENGTH_SHORT).show()
        R.id.radio_dos -> Snackbar.make(binding.root, "Dos", Snackbar.LENGTH_SHORT).show()
    }
}
```

5.3 Spinner con datos desde recursos

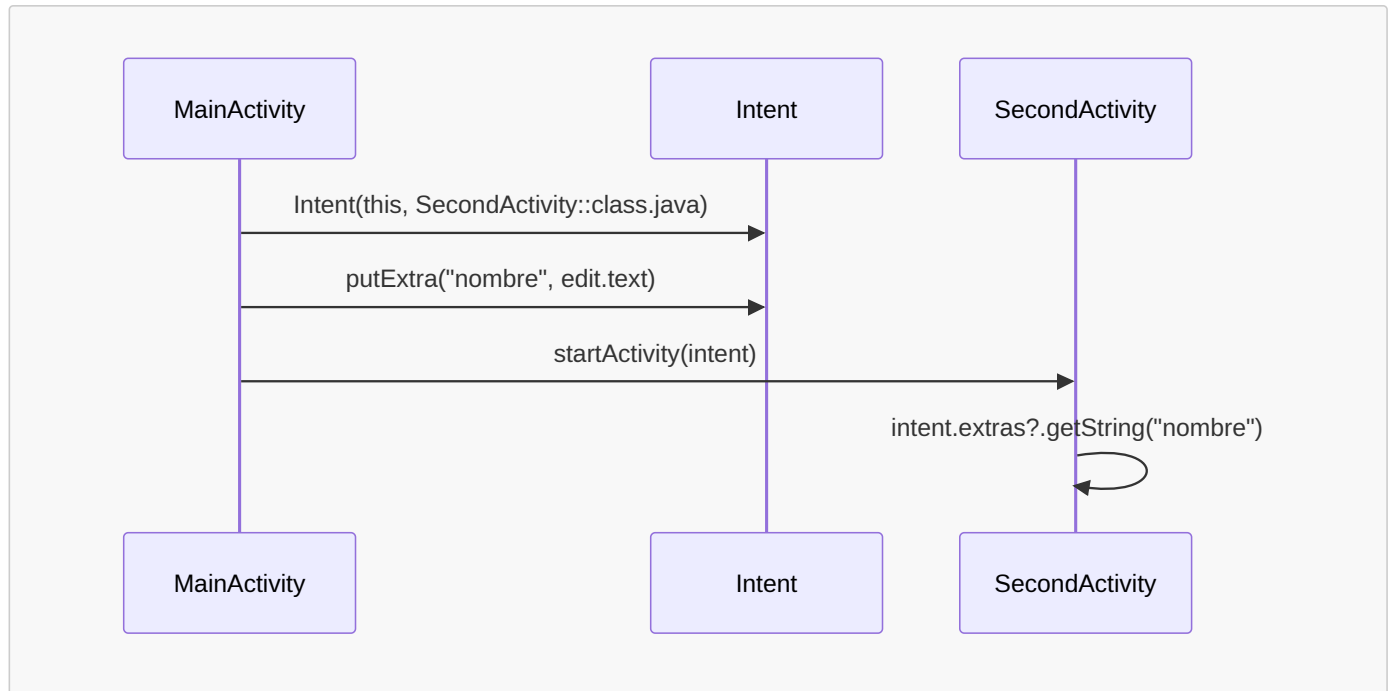
```
<string-array name="países">
    <item>Espana</item>
    <item>Italia</item>
    <item>Francia</item>
</string-array>
```

```
val adapter = ArrayAdapter.createFromResource(
    this,
    R.array.países,
    android.R.layout.simple_spinner_item
)
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
binding.spinnerSeleccion.adapter = adapter
```

6. Intents y navegacion Android

6.1 Tipos

Tipo	Uso	Ejemplo
Explicito	Abrir una activity concreta de la app.	<code>Intent(this, SecondActivity::class.java)</code>
Implicito	Pedir al sistema una accion.	<code>Intent(Intent.ACTION_VIEW, Uri.parse(url))</code>
Con extras	Pasar datos clave-valor.	<code>putExtra("nombre", valor)</code>
Con resultado	Abrir algo esperando respuesta.	Camara, selector de archivo, etc.



```

val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("nombre", binding.editNombre.text.toString())
startActivity(intent)

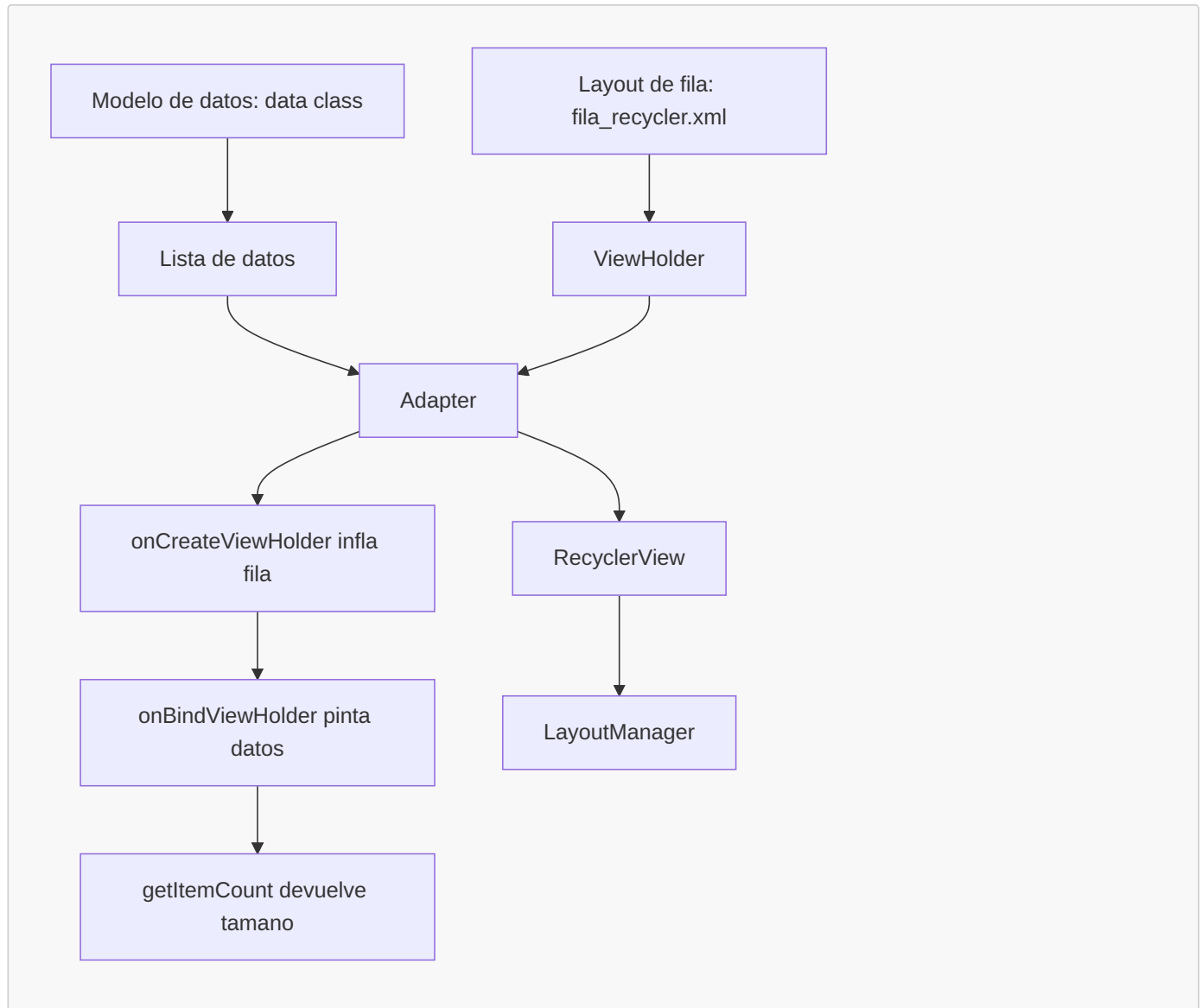
val nombre = intent.extras?.getString("nombre").orEmpty()
  
```

7. Listas en Android

7.1 ListView, Spinner y RecyclerView

Elemento	Cuando usar	Adaptador
Spinner	Selección de un elemento en desplegable.	ArrayAdapter.
ListView	Listas pequeñas o sencillas.	ArrayAdapter o BaseAdapter.
RecyclerView	Listas grandes, personalizadas y eficientes.	RecyclerView.Adapter.

7.2 Flujo de RecyclerView



Metodo del adapter	Funcion
<code>getItemCount()</code>	Devuelve cuantos elementos hay.
<code>onCreateViewHolder()</code>	Infla el layout de cada fila y crea el holder.
<code>onBindViewHolder()</code>	Asocia el dato de una posicion con las vistas del holder.
<code>notifyDataSetChanged()</code>	Notifica cambios generales de datos.

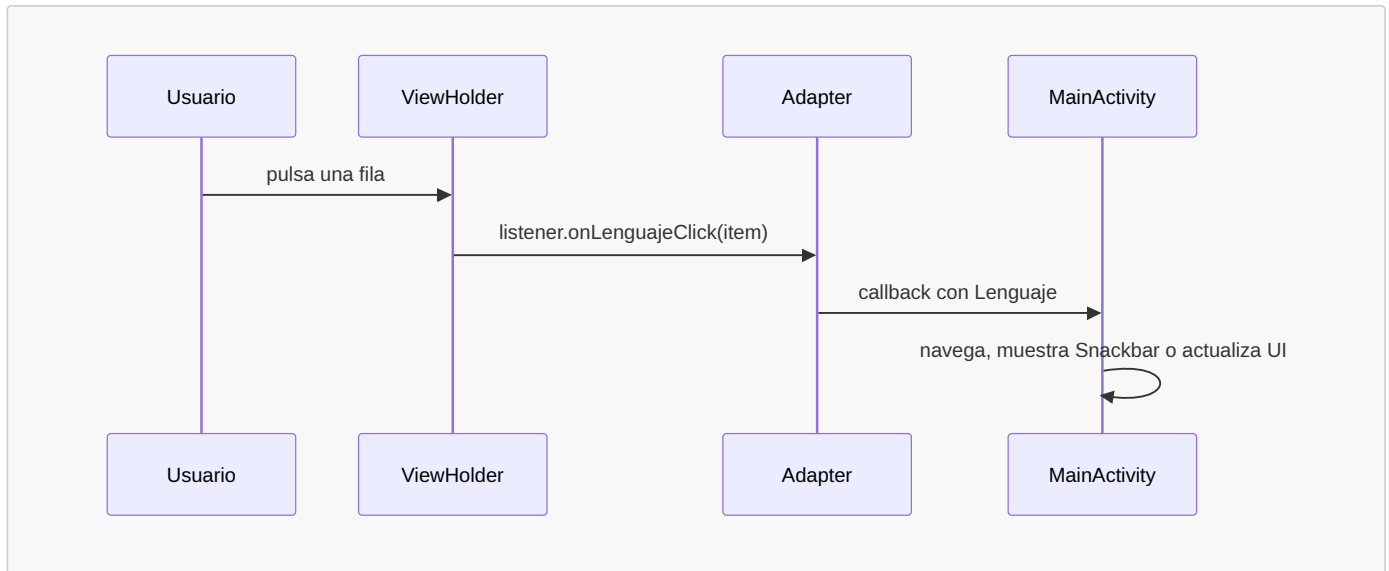
7.3 Eventos en RecyclerView

RecyclerView no tiene un `onItemClickListener` directo como `ListView`. Lo habitual es:

1. Capturar el click en el `ViewHolder` o en `onBindViewHolder`.
2. Comunicarlo a la `Activity` mediante una interfaz o lambda.

```

interface OnLenguajeListener {
    fun onLenguajeClick(lenguaje: Lenguaje)
}
  
```



8. Dialogos, menus, toolbar y notificaciones

8.1 DialogFragment

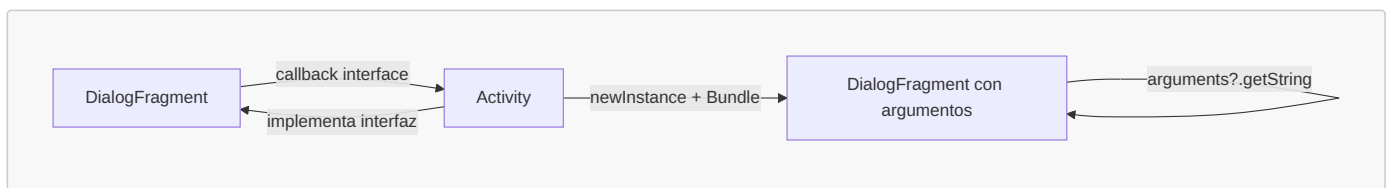
Tipo de dialogo	Metodo Builder	Uso
Aviso	<code>setTitle, setMessage, setPositiveButton</code>	Informar.
Confirmacion	<code>setPositiveButton, setNegativeButton, setNeutralButton</code>	Preguntar accion.
Seleccion	<code>setItems</code>	Elegir una opcion y cerrar.
Seleccion simple	<code>setSingleChoiceItems</code>	Elegir una opcion con radio.
Seleccion multiple	<code>setMultiChoiceItems</code>	Elegir varias opciones.
Personalizado	<code>setView(vista)</code>	Usar un XML propio.

```

class DialogAviso : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {
        return AlertDialog.Builder(requireContext())
            .setTitle("Aviso")
            .setMessage("Mensaje para el usuario")
            .setPositiveButton("OK", null)
            .create()
    }
}

DialogAviso().show(supportFragmentManager, "aviso")
  
```

8.2 Comunicacion con dialogos



Direccion	Tecnica	Clave
Dialogo -> Activity	Interfaz callback.	La activity implementa la interfaz y el dialogo llama al listener.
Dialogo -> Activity	Lambda nullable.	<code>funcionComunicar?.invoke(datos)</code> .
Activity -> Dialogo	<code>newInstance + Bundle</code> .	<code>dialog.arguments = bundle</code> .

8.3 Menus

Menu	Donde se define	Metodo Kotlin
Opciones	res/menu/menu_principal.xml	onCreateOptionsMenu
Pulsaciones	XML con android:id	onOptionsItemSelected
Contextual	res/menu/menu_contextual.xml	registerForContextMenu, onCreateContextMenu, onContextItemSelected

```

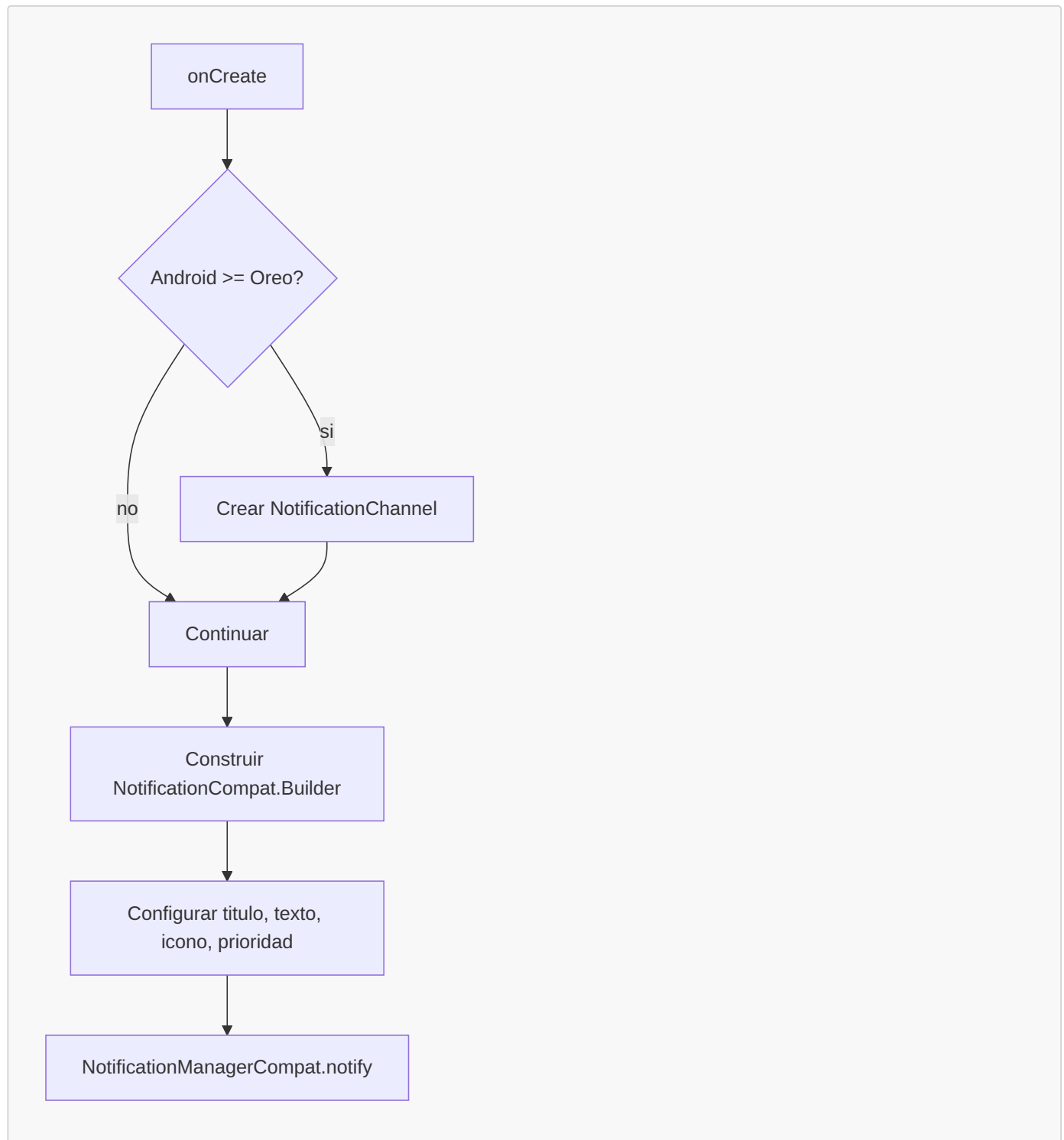
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.menu_principal, menu)
    return true
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when (item.itemId) {
        R.id.menu_salir -> finish()
    }
    return true
}

```

8.4 Toolbar y notificaciones

Elemento	Pasos
Toolbar	Usar tema NoActionBar, anadir Toolbar al XML, ejecutar <code>setSupportActionBar(binding.toolbar)</code> .
Boton home	<code>supportActionBar?.setDisplayHomeAsUpEnabled(true)</code> y gestionar <code>android.R.id.home</code> .
Notificaciones Android 8+	Crear <code>NotificationChannel</code> .
Lanzar notificacion	<code>NotificationCompat.Builder</code> , configurar titulo/texto/icono y <code>NotificationManagerCompat.notify(...)</code> .



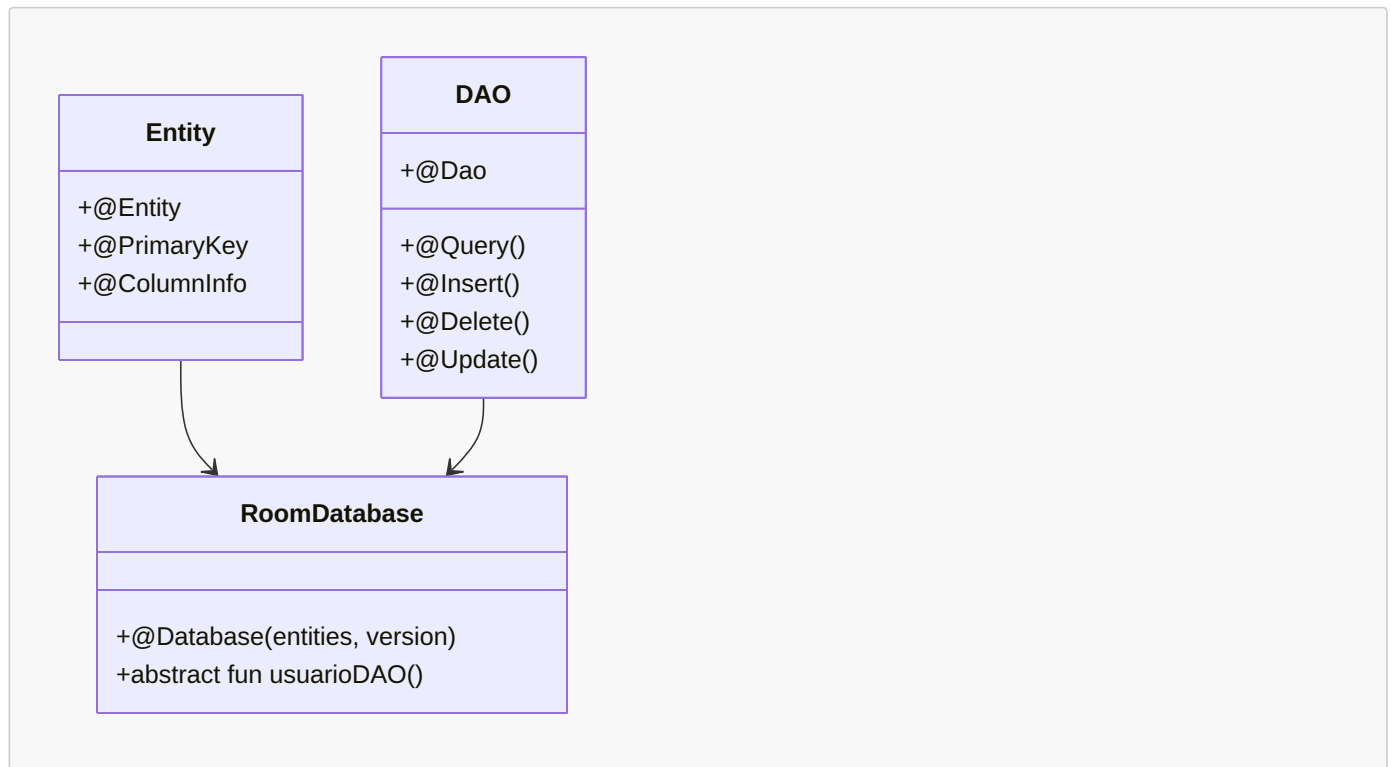
9. Bases de datos y Firebase

9.1 SQLite vs Room vs Firebase

Tecnologia	Tipo	Ventajas	Cuando usar
SQLite directo	Local relacional	Control total SQL.	Casos simples o aprendizaje de SQL Android.
Room	Local relacional con abstraccion	Menos boilerplate, anotaciones, DAO.	Persistencia local moderna.
Firebase Realtime Database	Remota NoSQL por nodos	Tiempo real, sencilla para datos pequenos/medios.	Apps sincronizadas y prototipos rapidos.
Firebase Auth	Autenticacion	Email/password y otros proveedores.	Login seguro sin backend propio.

Tecnologia	Tipo	Ventajas	Cuando usar
Firebase Storage	Archivos remotos	Imagenes, documentos, binarios.	Subida/descarga de recursos de usuario.

9.2 Room: componentes



Pieza	Representa	Ejemplo
<code>@Entity</code>	Tabla.	<code>data class Usuario(...)</code>
<code>@PrimaryKey</code>	Clave primaria.	<code>@PrimaryKey(autoGenerate = true)</code>
<code>@ColumnInfo</code>	Nombre/configuracion de columna.	<code>@ColumnInfo(name = "email")</code>
<code>@Dao</code>	Operaciones sobre tablas.	<code>selectAll, insert, delete.</code>
<code>@Database</code>	Base de datos completa.	<code>abstract class UsuariosBD : RoomDatabase()</code>

```

@Entity
data class Usuario(
    @ColumnInfo(name = "name") val nombre: String,
    @ColumnInfo(name = "email") val correo: String
) {
    @PrimaryKey(autoGenerate = true)
    var id: Long = 0
}

@Dao
interface UsuarioDao {
    @Query("SELECT * FROM Usuario")
    fun selectAll(): List<Usuario>

    @Insert
    fun insert(usuario: Usuario): Long
}
  
```

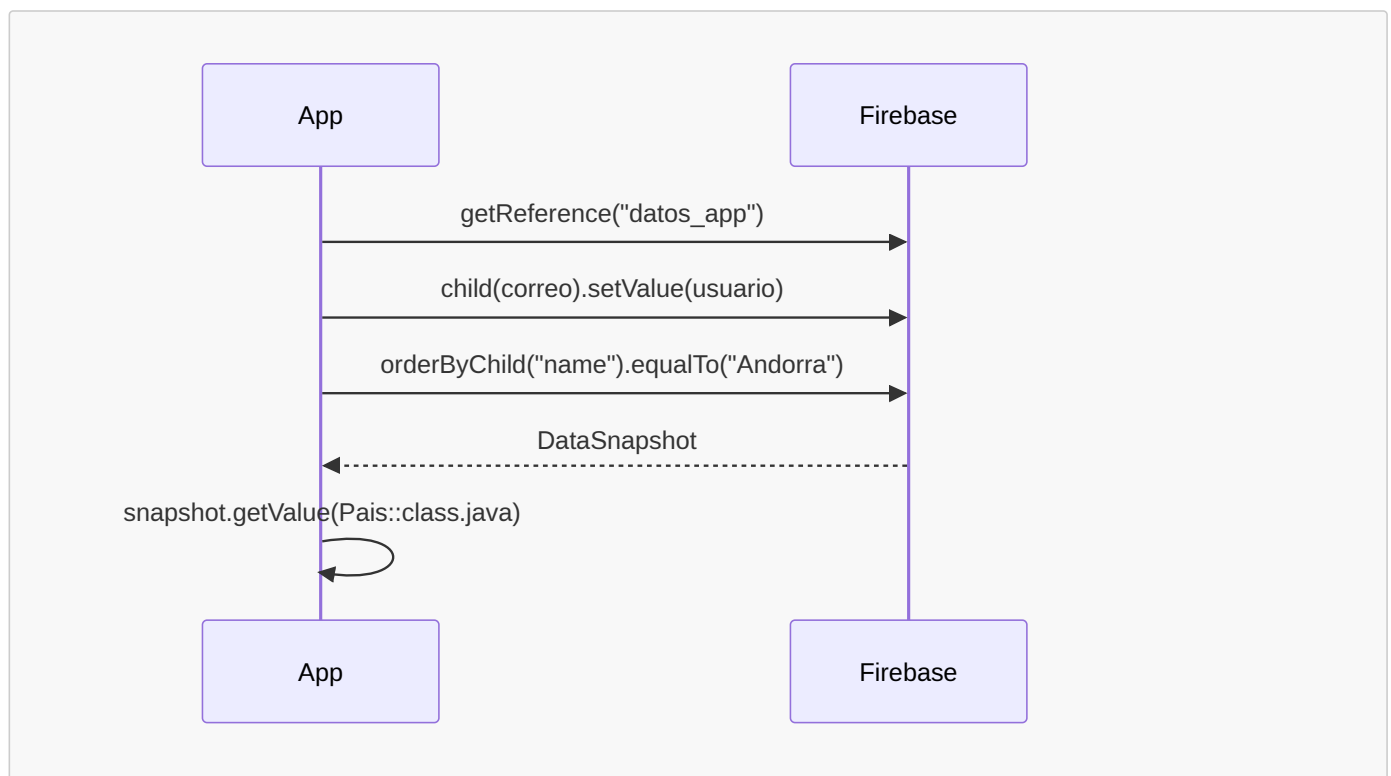
9.3 Corrutinas y Dispatchers

Dispatcher / Scope	Uso
<code>Dispatchers.Main</code>	UI: pintar, cambiar vistas, listeners.
<code>Dispatchers.IO</code>	Base de datos, red, ficheros.
<code>Dispatchers.Default</code>	Calculo intensivo.
<code>GlobalScope</code>	Vive mientras viva la app; usar con cuidado.
<code>lifecycleScope</code>	Asociado al ciclo de vida de activity/fragment.
<code>viewModelScope</code>	Asociado a un ViewModel.
<code>withContext(...)</code>	Cambiar de hilo dentro de una corrutina.

```
GlobalScope.launch(Dispatchers.IO) {
    val lista = database.usuarioDAO().selectAll()
    withContext(Dispatchers.Main) {
        adapter.changeList(lista)
    }
}
```

9.4 Firebase Realtime Database

Operacion	Codigo base
Acceder	<code>val database = Firebase.database(url)</code>
Referencia	<code>val ref = database.getReference("usuarios")</code>
Insertar/modificar	<code>ref.child(id).setValue(usuario)</code>
Borrar	<code>ref.child(id).setValue(null)</code>
Consultar	<code>orderByChild("name").equalTo("Andorra")</code>
Leer una vez	<code>addListenerForSingleValueEvent(...)</code>
Mapear objeto	<code>snapshot.getValue(Pais::class.java)</code>



9.5 Firebase Auth

Accion	Metodo
Obtener auth	<code>val auth = Firebase.auth</code>
Registrar	<code>createUserWithEmailAndPassword(email, password)</code>
Login	<code>signInWithEmailAndPassword(email, password)</code>
Usuario actual	<code>auth.currentUser</code>
Resultado asincrono	<code>addOnCompleteListener { task -> ... }</code>

```
auth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener { task ->
        if (task.isSuccessful) {
            val user = auth.currentUser
        }
    }
```

10. Dart

10.1 Sintaxis basica

Concepto	Dart	Comparacion mental
Punto de entrada	<code>void main() {}</code>	Similar a Kotlin/Java.
Mutable inferido	<code>var nombre = "Ana";</code>	Como <code>var</code> en Kotlin.
Constante en compilacion	<code>const PI = 3.14;</code>	Valor fijo desde compilacion.
Final en ejecucion	<code>final dni = calcular();</code>	No reasignable despues.
Nullable	<code>int? edad;</code>	Como <code>Int?</code> en Kotlin.
Forzar no nulo	<code>edad!</code>	Como <code>!!</code> en Kotlin.
Valor por defecto	<code>valor ?? 0</code>	Parecido a Elvis <code>?:</code> .
Inicializacion tardia	<code>late var listado;</code>	Como <code>lateinit</code> , con diferencias.

```
void main() {
    int? edad;
    print(edad ?? 0);

    late List<String> lenguajes;
    lenguajes = ["Dart", "Kotlin"];
}
```

10.2 Funciones en Dart

Tipo	Sintaxis
Funcion simple	<code>void saludar() {}</code>
Retorno	<code>int suma(int a, int b) => a + b;</code>
Parametros posicionales	<code>registrar("Ana", "DAM")</code>
Parametros opcionales	<code>void f(String n, [int edad = 0])</code>
Parametros nombrados	<code>void f({required String nombre})</code>
Funcion anonima	<code>(valor) => print(valor)</code>

```
void registrarUsuario({
  required String nombre,
  required String departamento,
  int salario = 0,
}) {
  print('$nombre - $departamento - $salario');
}
```

10.3 Colecciones Dart

Coleccion	Sintaxis	Operaciones
List	List<String> nombres = [];	add, remove, removeAt, firstWhere.
Lista fija	List.filled(5, null)	Tamano predefinido.
Map	Map<String, dynamic> usuario = {}	usuario['nombre'], addAll.
Recorrido	for, forEach, iterator	Segun necesidad.

```
final usuario = <String, dynamic>{
  'nombre': 'Ana',
  'edad': 20,
};

usuario.addAll({'curso': 'DAM'});
print(usuario['nombre']);
```

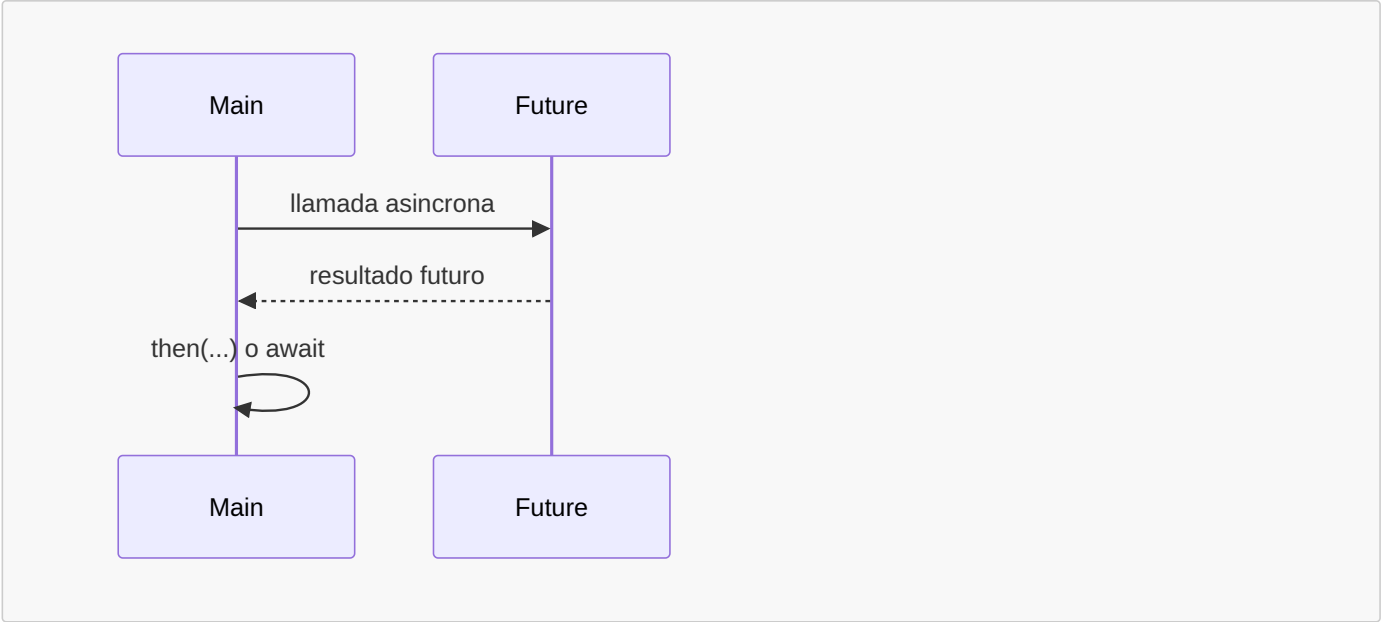
10.4 Clases, herencia y mixins

Concepto	Dart
Clase	class Usuario { ... }
Privado de libreria	Prefijo _: _dni
Constructor resumido	Usuario(this.nombre, this.edad);
Constructor con nombrados	Usuario({required this.nombre});
Constructor nombrado	Usuario.sinCorreo(...)
Getter / setter	String get nombre => _nombre;
Herencia	class Trabajador extends Usuario
Sobrescritura	@override
Mixin	class Vendedor extends Persona with Comercial

```
class Usuario {
  final String nombre;
  final String correo;

  Usuario({required this.nombre, required this.correo});
  Usuario.sinCorreo(this.nombre) : correo = '';
}
```

10.5 Asincronia



Elemento	Uso
<code>Future<T></code>	Resultado disponible en el futuro.
<code>.then(...)</code>	Gestiona resultado cuando llega.
<code>.catchError(...)</code>	Gestiona error.
<code>async</code>	Marca una funcion asincrona.
<code>await</code>	Espera un <code>Future</code> ; solo dentro de <code>async</code> .

```
Future<String> realizaTarea() async {
  final resultado = await Future(() => "Tarea realizada");
  return resultado;
}

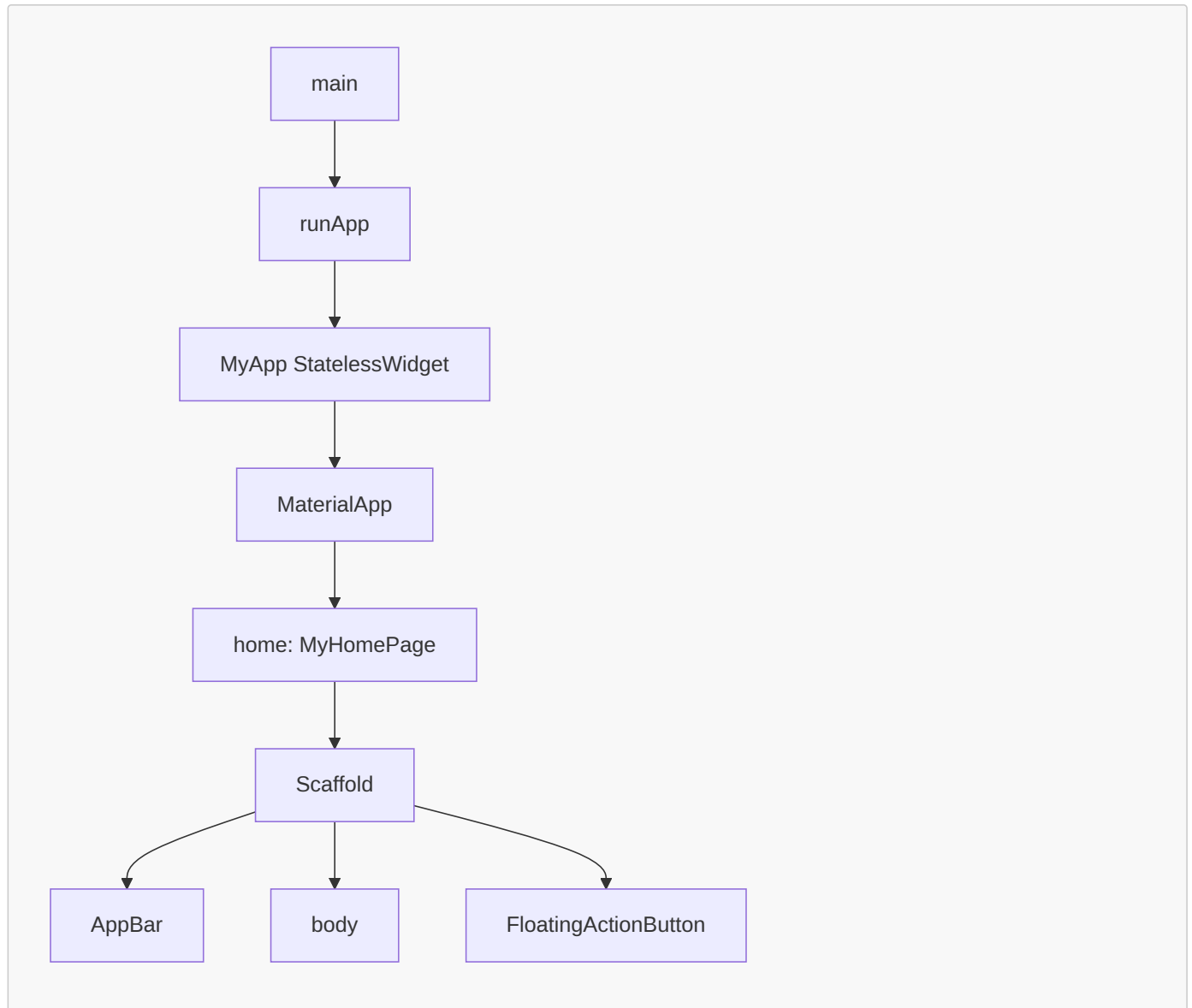
void main() async {
  print(await realizaTarea());
}
```

11. Flutter

11.1 Estructura de proyecto

Ruta	Contenido
<code>lib</code>	Codigo comun de la aplicacion.
<code>lib/main.dart</code>	Entrada principal.
<code>android, ios, web, windows, macos</code>	Proyectos especificos por plataforma.
<code>pubspec.yaml</code>	Dependencias, assets y configuracion.

11.2 Arbol basico Flutter



Widget	Funcion
<code>MaterialApp</code>	Configuracion global Material, tema, rutas, home.
<code>Scaffold</code>	Estructura de pantalla: <code>appBar</code> , <code>body</code> , <code>floatingActionButton</code> .
<code>AppBar</code>	Barra superior.
<code>Center</code>	Centra un hijo.
<code>Column / Row</code>	Colocan hijos vertical u horizontalmente.
<code>Container</code>	Caja con padding, color, tamaño, alineacion.
<code>Expanded</code>	Ocupa espacio disponible dentro de <code>Row/Column</code> .
<code>Table</code>	Organiza por filas y columnas.

11.3 StatelessWidget vs StatefulWidget

Tipo	Guarda estado	Metodo clave	Uso
<code>StatelessWidget</code>	No	<code>build(context)</code>	UI que depende solo de parametros.
<code>StatefulWidget</code>	Si	<code>createState() + State.build()</code>	Formularios, contadores, seleccion, cambios visuales.



Syntax error in text

mermaid version 11.15.0

```
class ContadorPage extends StatefulWidget {
  const ContadorPage({super.key});

  @override
  State<ContadorPage> createState() => _ContadorPageState();
}

class _ContadorPageState extends State<ContadorPage> {
  int contador = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(child: Text('$contador')),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => contador++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

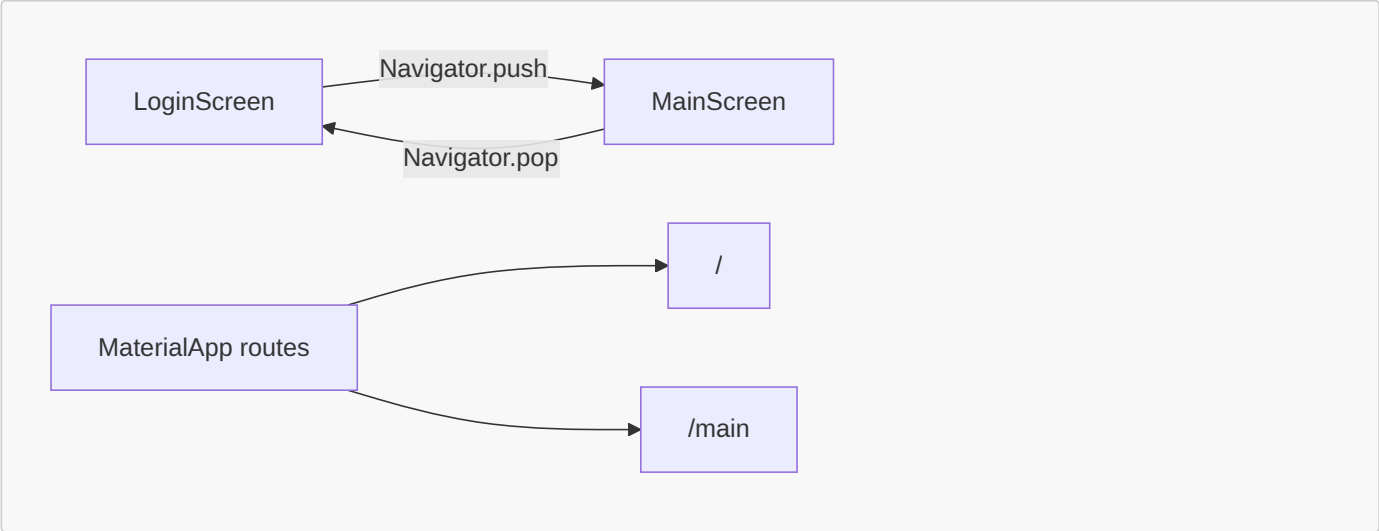
11.4 Widgets graficos y controles

Widget	Uso	Estado necesario
<code>ElevatedButton</code> , <code>FilledButton</code> , <code>OutlinedButton</code> , <code>TextButton</code>	Botones de accion.	No siempre.
<code>IconButton</code>	Boton con icono.	No siempre.
<code>FloatingActionButton</code>	Accion principal.	No siempre.
<code>SegmentedButton</code>	Seleccion entre opciones.	Si quieres mantener seleccion.
<code>Radio</code>	Una opcion dentro de grupo.	Si.
<code>Checkbox</code>	Booleano seleccionado/no seleccionado.	Si.
<code>TextField</code>	Entrada de texto.	Normalmente si, con <code>TextEditingController</code> .
<code>SnackBar</code>	Mensaje inferior temporal.	Se lanza con <code>ScaffoldMessenger</code> .

```
final TextEditingController controllerMail = TextEditingController();

TextField(
  controller: controllerMail,
  decoration: const InputDecoration(
    label: Text("Introduce tu correo"),
    border: OutlineInputBorder(),
    prefixIcon: Icon(Icons.mail),
  ),
)
```

11.5 Navegacion Flutter



Forma	Codigo	Uso
Navegacion directa	<code>Navigator.push(context, MaterialPageRoute(...))</code>	Pantallas simples.
Volver	<code>Navigator.pop(context)</code>	Regresar a la pantalla anterior.
Rutas nombradas	<code>routes: {'/main': (...) => MainPage() }</code>	Apps con varias pantallas.
Navegar por nombre	<code>Navigator.pushNamed(context, '/main')</code>	Rutas centralizadas.

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const MainPage()),
);

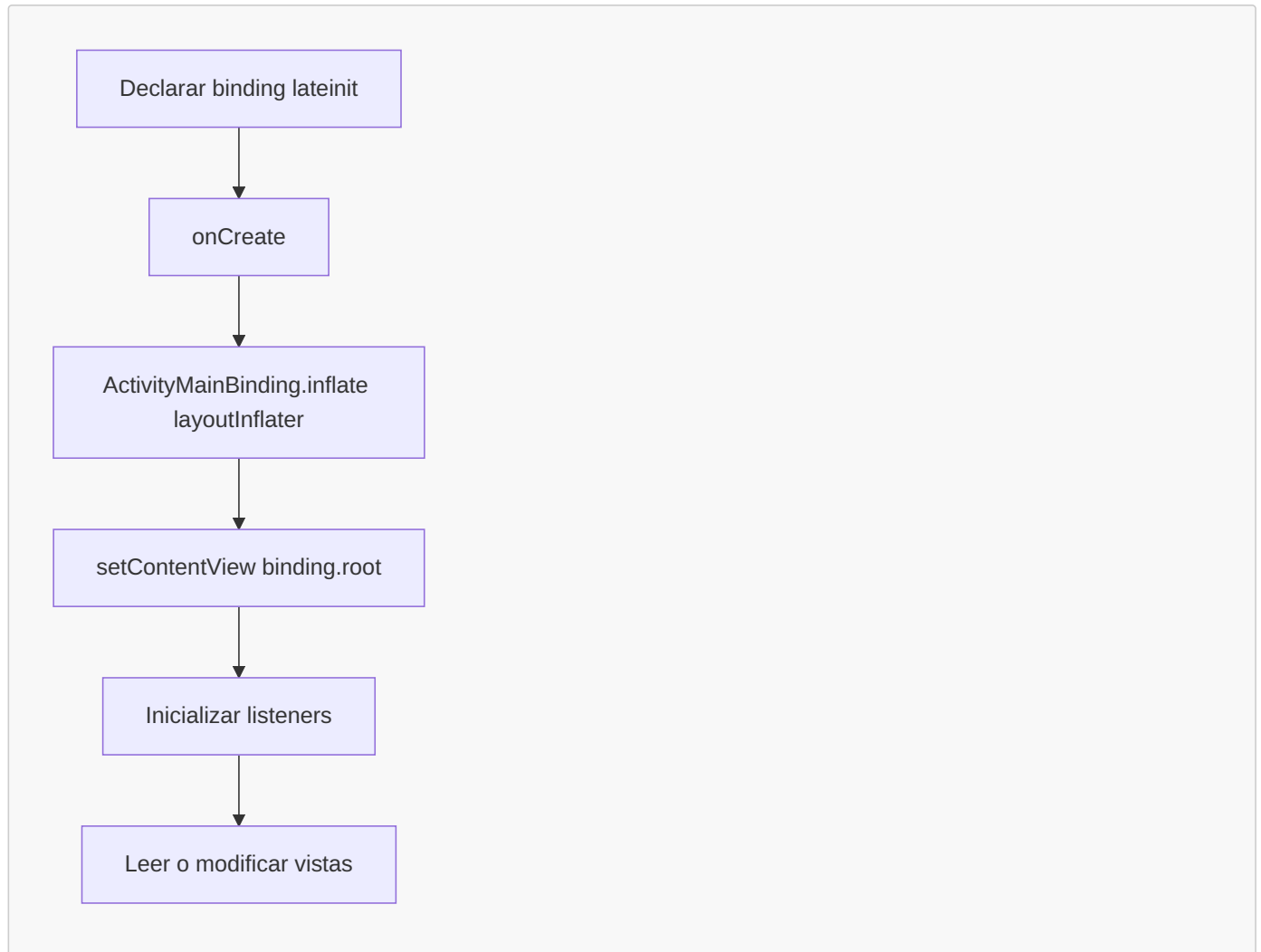
Navigator.pushNamed(context, '/main');
```

12. Chuleta rapida Kotlin vs Dart

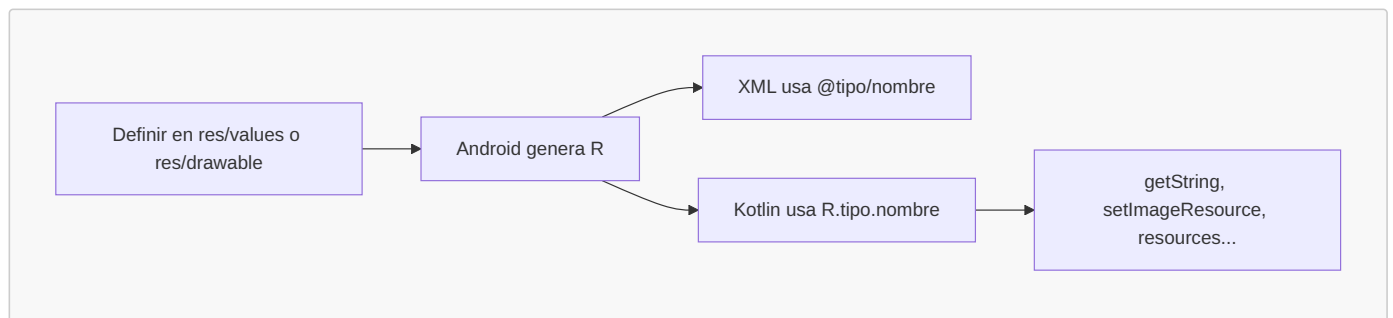
Tema	Kotlin	Dart
Funcion main	<code>fun main() {}</code>	<code>void main() {}</code>
Variable mutable	<code>var x = 1</code>	<code>var x = 1;</code>
Constante/referencia fija	<code>val x = 1</code>	<code>final x = 1; / const x = 1;</code>
Nullable	<code>String?</code>	<code>String?</code>
Llamada segura	<code>obj?.x</code>	<code>obj?.x</code>
Forzar no nulo	<code>obj!!</code>	<code>obj!</code>
Valor por defecto si null	<code>a ?: b</code>	<code>a ?? b</code>
Clase	<code>class Persona(val n: String)</code>	<code>class Persona { Persona(this.n); }</code>
Herencia	<code>class Hija : Padre()</code>	<code>class Hija extends Padre</code>
Interfaz/mixins	<code>interface</code>	<code>abstract class, mixin, with</code>
Lambda	<code>{ x: Int -> x + 1 }</code>	<code>(x) => x + 1</code>
Asincronia	Corrutinas: <code>launch, withContext</code>	<code>Future, async, await</code>

13. Flujos que conviene memorizar

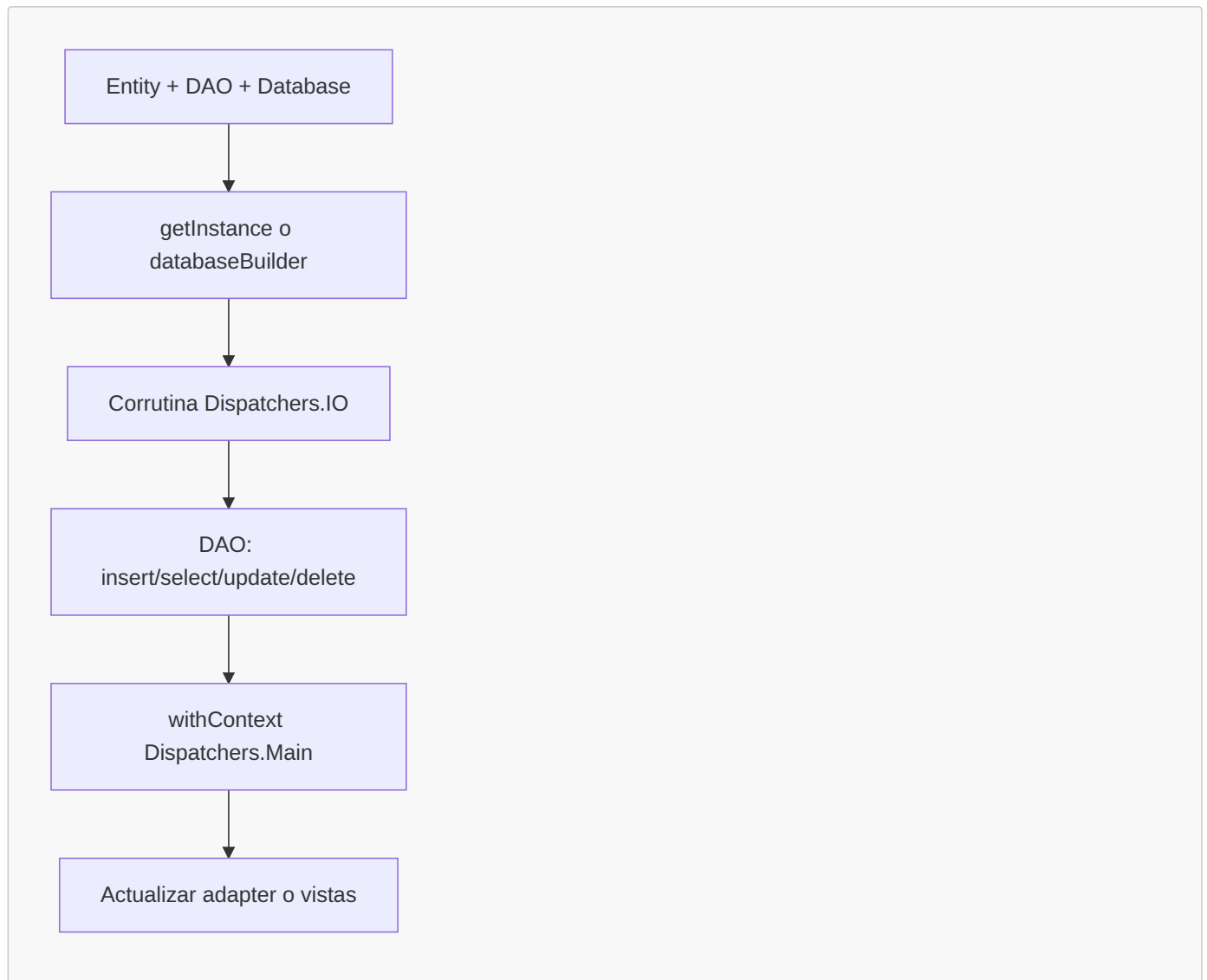
Activity con ViewBinding



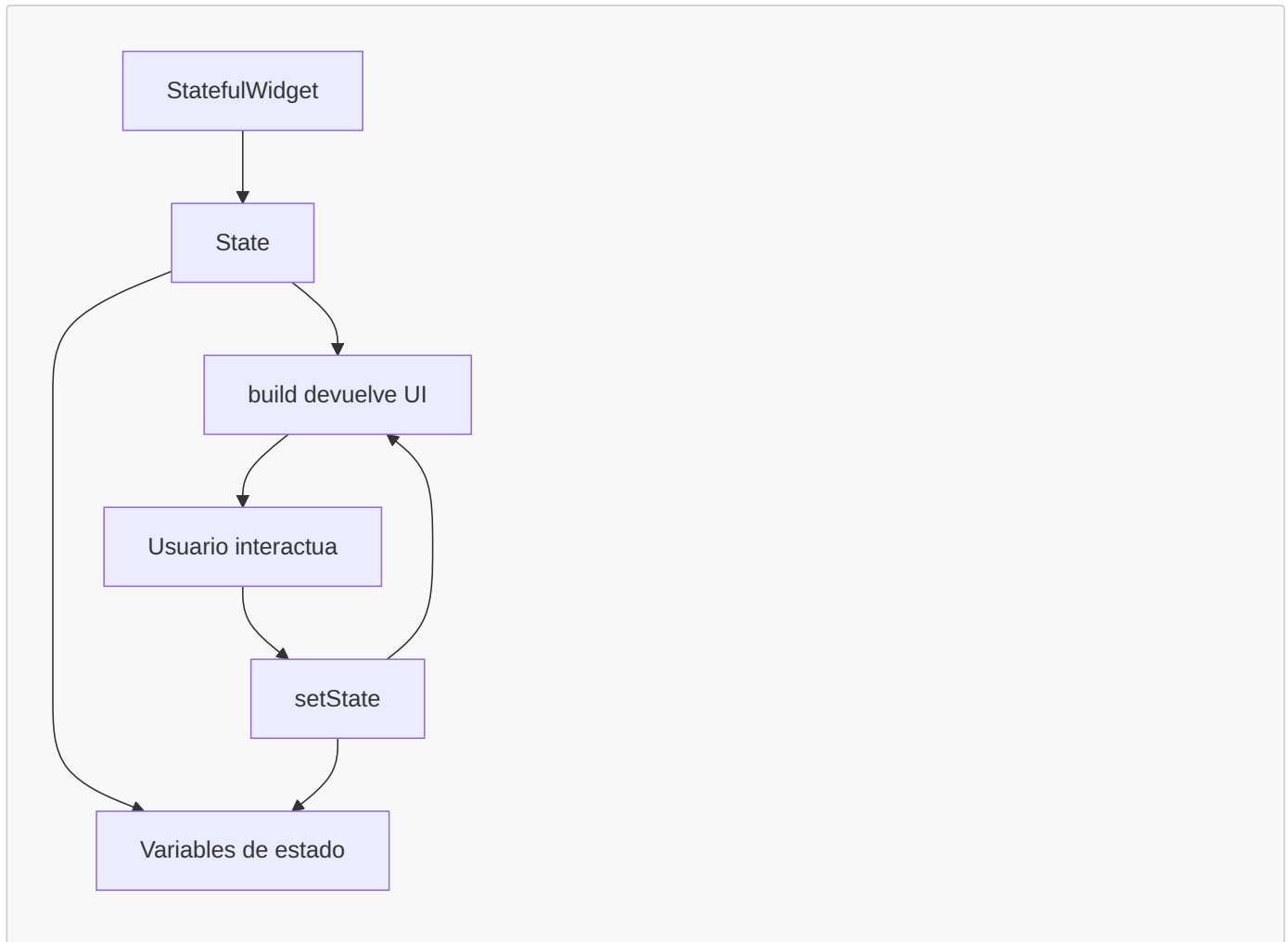
Crear un recurso y usarlo



Room con UI



Flutter con estado

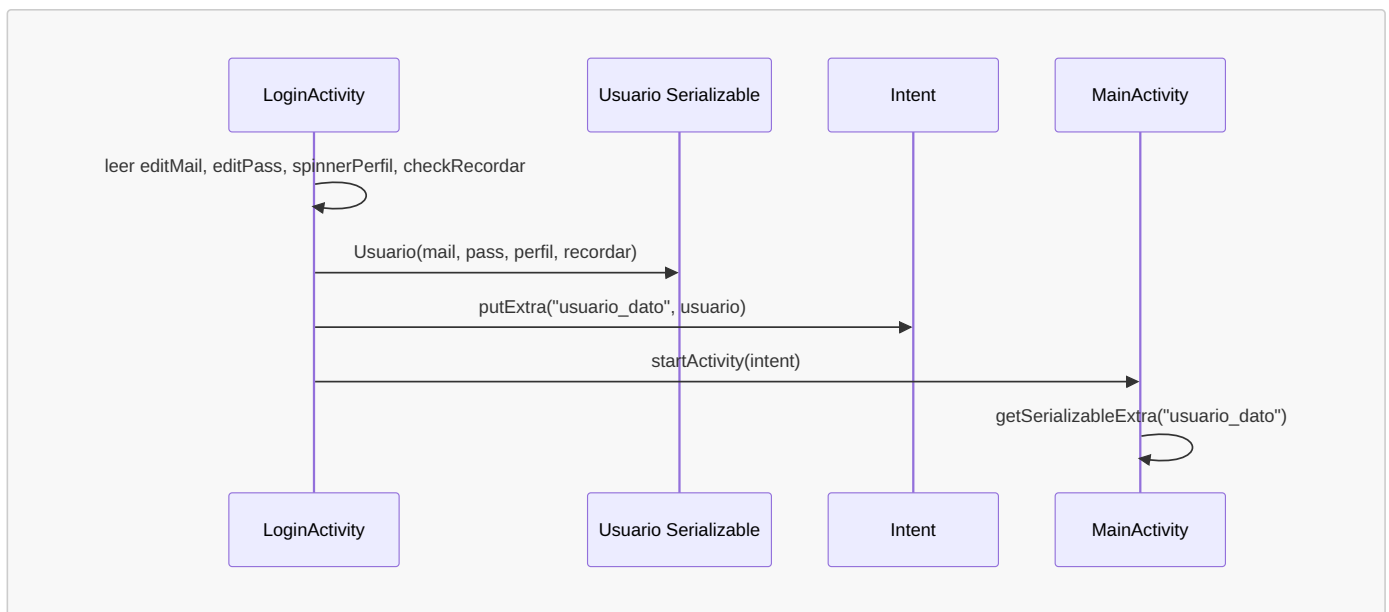


14. Ejemplos adaptados del codigo de clase

Basado en [DevelopSys/CodigosUTAD](#), ruta [2526/MAISFII](#): proyectos [TEMA2/inicio](#), [TEMA3/inicio](#), [TEMA4/inicio](#), [TEMA5/TiendaFragments](#), [TEMA6/TiendaFragments](#) y [TEMA7/Calculadora](#). Los ejemplos estan compactados y adaptados para estudiar.

14.1 Login Android: XML + ViewBinding + Spinner + Intent

El flujo de clase es: pantalla de login, captura de correo/pass/perfil/recordar, creacion de un objeto [Usuario](#) y navegacion a [MainActivity](#).



Pieza	Responsabilidad	Ejemplo adaptado
activity_login.xml	Define <code>EditText</code> , <code>Spinner</code> , <code>CheckBox</code> y botones.	<code>@+id/editMail, @+id/spinnerPerfil.</code>
ActivityLoginBinding	Accede a las vistas sin <code>findViewById</code> .	<code>binding.editMail.text.toString().</code>
ArrayAdapter	Alimenta el <code>Spinner</code> .	<code>ArrayAdapter(this, simple_spinner_item, perfiles).</code>
Usuario : Serializable	Permite pasar un objeto por <code>Intent</code> .	<code>intent.putExtra("usuario_dato", usuario).</code>
<code>onRestart()</code>	Limpia el formulario al volver.	<code>binding.editPass.text.clear().</code>

```
data class Usuario(
    val mail: String,
    val pass: String,
    val perfil: String,
    val recordar: Boolean
) : Serializable

private fun configurarSpinner() {
    val perfiles = listOf("Administrador", "Usuario", "Invitado")
    val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_item, perfiles)
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
    binding.spinnerPerfil.adapter = adapter
}

private fun hacerLogin() {
    val usuario = Usuario(
        mail = binding.editMail.text.toString(),
        pass = binding.editPass.text.toString(),
        perfil = binding.spinnerPerfil.selectedItem.toString(),
        recordar = binding.checkRecordar.isChecked
    )

    val intent = Intent(this, MainActivity::class.java)
    intent.putExtra("usuario_dato", usuario)
    startActivity(intent)
}
```

En la `Activity` destino:

```
private lateinit var usuario: Usuario

private fun getDatos() {
    usuario = intent.getSerializableExtra("usuario_dato") as Usuario
    binding.editNombre.setText(usuario.mail)
}
```

14.2 Activity con varios botones y ciclo de vida

El ejemplo de clase usa `View.OnClickListener` para centralizar los clicks y registra metodos del ciclo de vida con `Log.v`.

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        binding.botonSaludar.setOnClickListener(this)
    }
}
```

```

        binding.botonVaciar.setOnClickListener(this)
        binding.botonSalir?.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        when (v?.id) {
            binding.botonVaciar.id -> binding.editNombre.text.clear()
            binding.botonSaludar.id -> saludar()
            binding.botonSalir?.id -> finish()
        }
    }

    override fun onStart() {
        super.onStart()
        Log.v("ciclo_vida", "onStart")
    }
}

```

Detalle importante	Por que importa
<code>binding.botonSalir?.setOnClickListener(this)</code>	El boton puede existir solo en <code>layout-land</code> ; el <code>?</code> evita fallo en vertical.
<code>when (v?.id)</code>	Permite que varios botones compartan un unico listener.
<code>resources.getString(R.string.text_empty_edit)</code>	Lee un recurso desde Kotlin, en vez de hardcodear texto.

14.3 RecyclerView de productos: lista local y cambio por orientacion

En **TEMA3** se usa una lista local de productos. La pantalla decide el `LayoutManager` segun la orientacion.

```

private fun initGUI() {
    binding.recyclerProductos.adapter = adapterProduct

    binding.recyclerProductos.layoutManager =
        if (resources.configuration.orientation == Configuration.ORIENTATION_PORTRAIT) {
            LinearLayoutManager(this, LinearLayoutManager.VERTICAL, false)
        } else {
            GridLayoutManager(this, 2, LinearLayoutManager.VERTICAL, false)
        }
}

```

Adapter con `ViewBinding` en la fila:

```

class ProductAdapter(
    private val list: ArrayList<Product>,
    private val context: Context
) : RecyclerView.Adapter<ProductAdapter.ProductHolder>() {

    inner class ProductHolder(val binding: ItemProductoBinding) :
        RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ProductHolder {
        val binding = ItemProductoBinding.inflate(LayoutInflater.from(context), parent, false)
        return ProductHolder(binding)
    }

    override fun onBindViewHolder(holder: ProductHolder, position: Int) {
        val product = list[position]
        holder.binding.textoProducto.text = product.title
    }

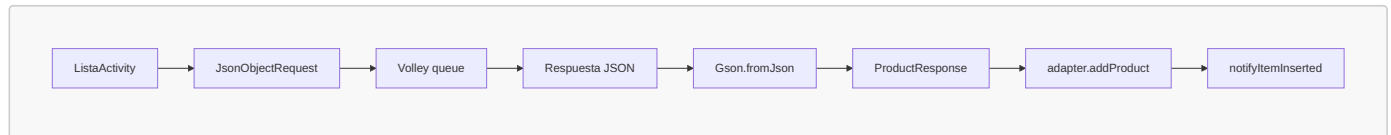
    override fun getItemCount(): Int = list.size
}

```

Metodo	En este ejemplo se usa para
<code>onCreateViewHolder</code>	Inflar <code>item_producto.xml</code> .
<code>onBindViewHolder</code>	Pintar el titulo del producto en la fila.
<code>getItemCount</code>	Indicar cuantos productos hay.
<code>LinearLayoutManager</code>	Lista vertical en portrait.
<code>GridLayoutManager</code>	Parrilla de 2 columnas en landscape.

14.4 RecyclerView con API: Volley + Gson + Glide

En **TEMA4** el listado deja de depender de datos locales y carga productos desde <https://dummyjson.com/products>.



```

private fun getJSONData() {
    val request = JsonObjectRequest(
        "https://dummyjson.com/products",
        { response ->
            val gson = Gson()
            val productResponse = gson.fromJson(response.toString(), ProductResponse::class.java)
            productResponse.products.orEmpty().forEach { adapterProduct.addProduct(it) }
        },
        { error ->
            Snackbar.make(binding.root, "Error cargando productos", Snackbar.LENGTH_LONG).show()
        }
    )

    Volley.newRequestQueue(this).add(request)
}
  
```

Adapter con lista interna e imagen remota:

```

class ProductAdapter(private val context: Context) :
    RecyclerView.Adapter<ProductAdapter.ProductHolder>() {

    private val list = ArrayList<Product>()

    override fun onBindViewHolder(holder: ProductHolder, position: Int) {
        val product = list[position]
        holder.binding.textoProducto.text = product.title

        Glide.with(context)
            .load(product.thumbnail)
            .placeholder(R.drawable.image_def)
            .into(holder.binding.imagenProducto)
    }

    fun addProduct(product: Product) {
        list.add(product)
        notifyItemInserted(list.size - 1)
    }
}
  
```

Libreria	Uso en el ejemplo
Volley	Ejecutar la petición HTTP y recibir JSON.

Libreria	Uso en el ejemplo
Gson	Convertir JSON a <code>ProductResponse</code> y <code>Product</code> .
Glide	Cargar <code>thumbnail</code> en un <code>ImageView</code> .
<code>notifyItemInserted</code>	Actualizar solo la fila nueva, mas preciso que refrescar todo.

14.5 Fragments con Navigation Component y argumentos

En **TEMA5** la **Activity** aloja un **NavHostFragment**; las pantallas reales son fragments.

```
<fragment
    android:id="@+id/nav_host_fragment_content_main"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph" />
```

`nav_graph.xml` define destino inicial, fragments y argumentos:

```
<navigation
    app:startDestination="@id/loginFragment">

    <fragment
        android:id="@+id/loginFragment"
        android:name="com.example.tiendafragments.ui.fragment.LoginFragment">
        <action
            android:id="@+id/action_loginFragment_to_registerFragment"
            app:destination="@id/registerFragment" />
        </fragment>

        <fragment
            android:id="@+id/registerFragment"
            android:name="com.example.tiendafragments.ui.fragment.RegisterFragment">
            <argument android:name="name" app:argType="string" />
            <argument android:name="pass" app:argType="string" />
        </fragment>
    </navigation>
```

Navegacion desde login:

```
binding.btnRegistro.setOnClickListener {
    val bundle = Bundle()
    bundle.putString("name", binding.editCorreo.text.toString())
    bundle.putString("pass", binding.editPass.text.toString())
    findNavController().navigate(R.id.action_loginFragment_to_registerFragment, bundle)
}
```

Recuperacion en registro:

```
private lateinit var nombre: String
private lateinit var pass: String

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    nombre = requireArguments().getString("name").orEmpty()
    pass = requireArguments().getString("pass").orEmpty()
}
```

```

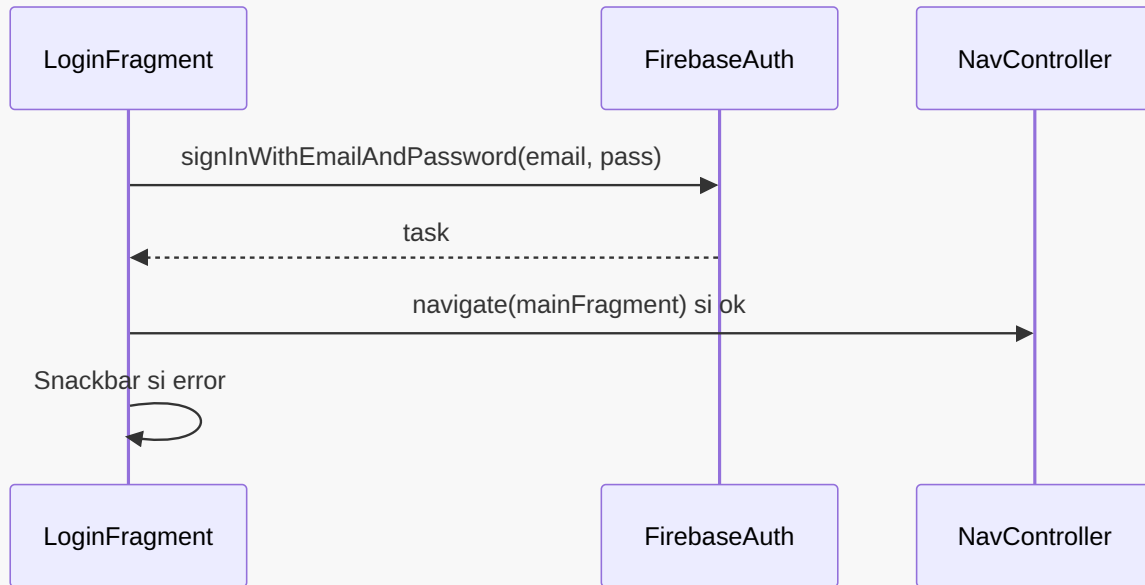
override fun onStart() {
    super.onStart()
    binding.editCorreo.setText(nombre)
    binding.editPass.setText(pass)
}

```

Mejora recomendable: en fragments, limpiar binding en `onDestroyView`. El código de clase usa `lateinit var binding;` para proyectos mas grandes es mejor `_binding: FragmentLoginBinding?`.

14.6 Firebase Auth en fragments

En **TEMA6** se reutiliza la navegacion con fragments y se anade **FirebaseAuth**.



```

private lateinit var auth: FirebaseAuth

override fun onAttach(context: Context) {
    super.onAttach(context)
    auth = FirebaseAuth.getInstance()
}

override fun onResume() {
    super.onResume()
    if (auth.currentUser != null) auth.signOut()
}

private fun login() {
    val email = binding.editCorreo.text.toString()
    val pass = binding.editPass.text.toString()

    auth.signInWithEmailAndPassword(email, pass)
        .addOnCompleteListener { task ->
            if (task.isSuccessful) {
                findNavController().navigate(R.id.action_loginFragment_to_mainFragment)
            } else {
                Snackbar.make(binding.root, "Error en el login", Snackbar.LENGTH_LONG).show()
            }
        }
}

```

Registro:

```

auth.createUserWithEmailAndPassword(correo, pass)
  .addOnCompleteListener { task ->
    if (task.isSuccessful) {
      Snackbar.make(binding.root, "Registro correcto", Snackbar.LENGTH_LONG)
        .setAction("OK") {
          findNavController().navigate(R.id.action_registerFragment_to_mainFragment)
        }
      .show()
    } else {
      Snackbar.make(binding.root, "Fallo en el registro", Snackbar.LENGTH_LONG).show()
    }
  }
}

```

Usuario actual:

```

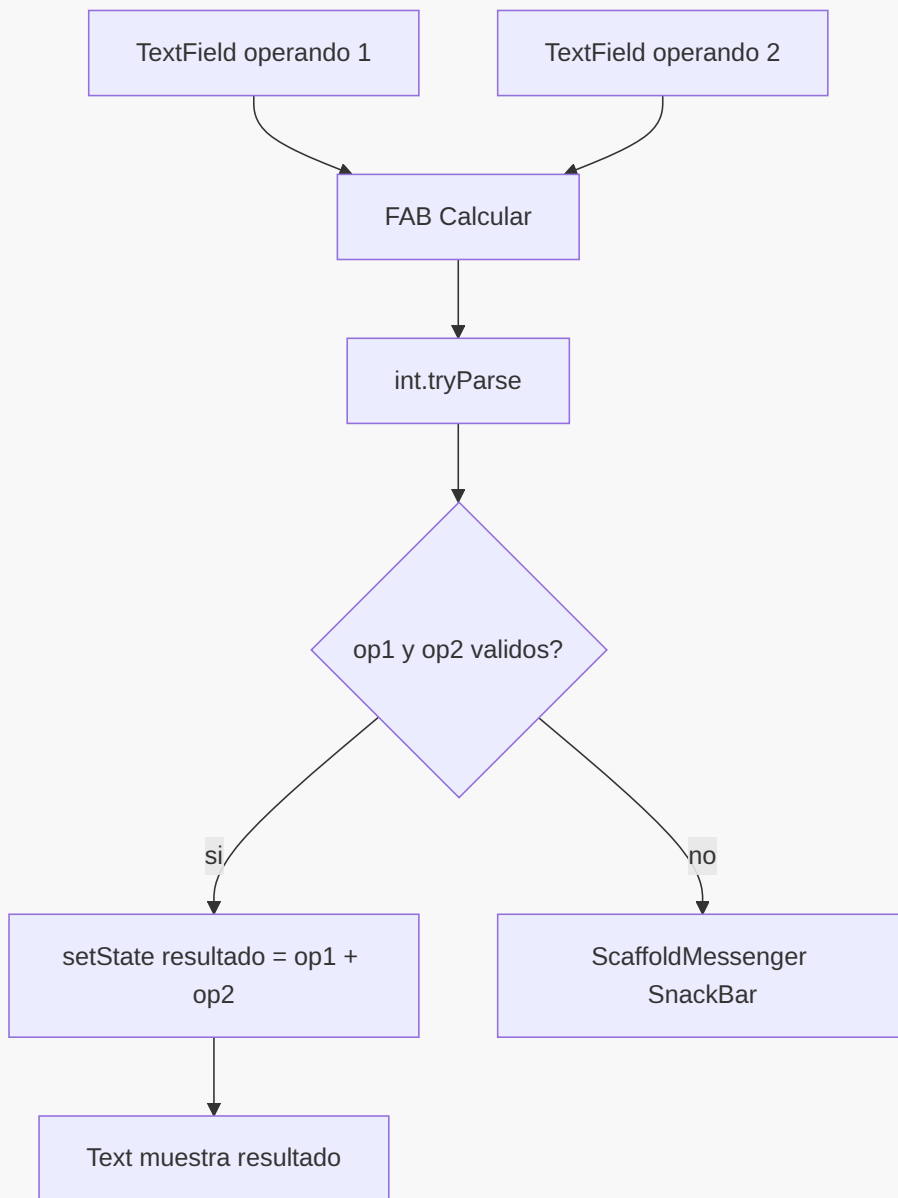
val currentUser = FirebaseAuth.getInstance().currentUser
binding.textMain.text = "Bienvenido ${currentUser?.uid ?: "sin usuario"}"

```

Punto de cuidado	Mejor practica
<code>currentUser!!</code>	Evitarlo salvo que se garantice usuario autenticado. Mejor <code>currentUser?.uid</code> .
Instanciar Firebase en fragment	Puede hacerse en <code>onAttach</code> u <code>onCreate</code> .
Operaciones Auth	Son asincronas; la UI se actualiza dentro del listener.

14.7 Flutter: calculadora con `StatefulWidget`

El proyecto `TEMA7/Calculadora` combina sintaxis Dart y Flutter real: variables `final`, `const`, nullable, constructores, `TextEditingController`, `tryParse`, `setState` y `SnackBar`.



```

class CalculadoraPage extends StatefulWidget {
  final String nombreAPP;

  const CalculadoraPage({super.key, required this.nombreAPP});

  @override
  State<CalculadoraPage> createState() => _CalculadoraState();
}

class _CalculadoraState extends State<CalculadoraPage> {
  int resultado = 0;
  final editOP1 = TextEditingController();
  final editOP2 = TextEditingController();

  void calcular() {
    final op1 = int.tryParse(editOP1.text);
    final op2 = int.tryParse(editOP2.text);

    if (op1 != null && op2 != null) {
      setState(() {
        resultado = op1 + op2;
      });
    } else {
  
```

```

        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text("Los datos son incorrectos")),
        );
    }
}
}

```

Concepto	Aparece en la calculadora como
<code>StatefulWidget</code>	La pantalla necesita guardar <code>resultado</code> .
<code>widget.nombreAPP</code>	El <code>State</code> lee propiedades del widget padre.
<code>TextEditingController</code>	Captura el contenido de cada <code>TextField</code> .
<code>int.tryParse</code>	Convierte texto a numero sin lanzar excepcion si falla.
<code>setState</code>	Cambia <code>resultado</code> y reconstruye la UI.
<code>ScaffoldMessenger</code>	Muestra un <code>SnackBar</code> si los datos no son validos.

15. Errores tipicos de examen/practica

Error	Por que falla	Solucion
Usar <code>findViewById</code> antes de <code>setContentview</code> .	La vista aun no existe.	Primero inflar layout, despues buscar vistas.
Hardcodear textos en XML/Kotlin.	Dificulta cambios e internacionalizacion.	Usar <code>strings.xml</code> y <code>@string/...</code>
Usar <code>!!</code> para todo.	Puede romper con <code>NullPointerException</code> .	Preferir <code>?.</code> y <code>?:</code> .
Actualizar UI desde <code>Dispatchers.IO</code> .	La UI debe tocarse en hilo principal.	Cambiar con <code>withContext(Dispatchers.Main)</code> .
Olvidar <code>layoutManager</code> en <code>RecyclerView</code> .	No sabe como colocar items.	<code>LinearLayoutManager(...)</code> .
No limpiar binding en Fragment.	Posibles fugas de memoria.	<code>_binding = null</code> en <code>onDestroyView</code> .
No crear canal de notificacion en Android 8+.	La notificacion no aparece.	Crear <code>NotificationChannel</code> .
Flutter: cambiar variable sin <code>setState</code> .	La UI no se reconstruye.	Envolver cambio en <code>setState</code> .
Flutter: usar <code>TextField</code> sin controller si necesitas leerlo.	No tienes acceso comodo al texto.	<code>TextEditingController</code> .
Dart: confundir <code>final</code> y <code>const</code> .	<code>const</code> es tiempo de compilacion; <code>final</code> en ejecucion.	Usar <code>final</code> cuando el valor se calcula al ejecutar.